Team ID: 26FSPDSCPMID4098

Case-Study Report

On "Resilient Data Science Programming for Fault-Tolerant Distributed Analytics in Multi-Cluster Spark Environments"

submitted as part of

Programming for Data Science (MACSE502)

by



REG.NO:26MML0054
NAME:MUDAVATH NARASIMHA NAIK



REG.NO:26MML0006
NAME:VANA SHIVARAMAKRISHNA KUMAR



REG.NO:26MML0047
NAME:VINAYAK C



REG.NO:26MML0015
NAME:K V KARTIK

To



Dr M Rajasekhara Babu

School of Computer Science and Engineering

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

Index

Sl. No.	Section Topic	Page No.
	Executive Summary and Abstract	6
1.	I. Introduction	7
2.	II. Problem Statement	7
3.	III. Background and Literature Review	8
4.	IV. Requirements Analysis for Resilient Analytics Systems	11
5.	V. Multi-Cluster Spark Analytics Architecture	12
6.	VI. Distributed Data Engineering Layer	14
7.	VII. Spark Processing and Execution Layer	15
8.	VIII. Fault Detection and Monitoring Framework	16
9.	IX. Fault-Tolerant Recovery and Resilience Layer	17
10.	X. Multi-Cluster Coordination and Orchestration Layer	19
11.	XI. Explainability and Reliability Assessment Layer	20
12.	XII. Security, Governance, and Compliance Layer	22
13.	XIII. Experimental Setup	23
14.	XIV. Evaluation Metrics	24
15.	XV. Results and Findings	26
16.	XVI. Comparative Analysis	28
17.	XVII. Reliability and Business Continuity Assessment	29
18.	XVIII. Industry Use Cases and Case Study Evaluation	30
19.	XIX. Business Impact Assessment	31
20.	XX. Challenges and Limitations	33
21.	XXI. Future Research Directions	33
22.	XXII. Conclusion	34
	Appendix	35
	References	35

List of Figures

Fig. No.	Figure Title	Page No.
1.	Evolution of Distributed Analytics Platforms	7
2.	Resilient Data Science Ecosystem	7
3.	Failure Scenarios in Distributed Analytics	8
4.	Literature Taxonomy Framework	9
5.	Requirements Engineering Framework	11
6.	Resilient Multi-Cluster Spark Architecture	13
7.	Layered Analytics Stack	13
8.	Distributed Data Engineering Pipeline	14
9.	Spark Execution Architecture	15
10.	DAG Processing Workflow	15
11.	Monitoring and Observability Architecture	16
12.	Fault Detection Workflow	17
13.	Fault Recovery Architecture	18
14.	Failover and Recovery Workflow	18
15.	Multi-Cluster Coordination Model	19
16.	Workload Orchestration Framework	19
17.	Explainable Reliability Framework	20
18.	Security and Governance Architecture	22
19.	Experimental Testbed Architecture	23
20.	Evaluation Framework	24
21.	Fault Recovery Performance	26
22.	Availability and Uptime Evaluation	27
23.	Resource Utilization and Cluster Stability	28
24.	Comparative Benchmark Analysis	28
25.	Business Continuity Framework	29
26.	Financial Analytics Resilience Architecture	30
27.	Smart Manufacturing Analytics Platform	31
28.	Business Impact Dashboard	32
29.	Challenges and Limitations Roadmap	33
30.	Future Research Roadmap	33
31.	Next-Generation Resilient Analytics Architecture	34
32.	Research Contribution Framework	34

List of Tables

Table. No.	Table Title	Page No.
1.	Executive summary of contributions and outcomes	6
2.	Research challenges and gaps	8
3.	Comparative analysis of existing approaches	11
4.	Functional and non-functional requirements matrix	12
5.	Data engineering components and responsibilities	15
6.	Monitoring metrics and thresholds	17
7.	Reliability assessment indicators	21
8.	Governance and compliance controls	23
9.	Experimental configuration details	24
10.	Metrics definition matrix	25
11.	Experimental results summary	27
12.	Comparative benchmark analysis	28
13.	Reliability and continuity assessment matrix	30
14.	Industry use cases matrix	31
15.	Cost-benefit analysis	32
16.	Recovery rules per mode	35

Contributions

Sl. No.	Section Topic	Reg. No. of Team Member contributed	No. of Words
	Executive Summary, Abstract and Keywords	26MML0015	376
1.	I. Introduction	26MML0047	586
2.	II. Problem Statement	26MML0006	296
3.	III. Background and Literature Review	26MML0047	1120
4.	IV. Requirements Analysis for Resilient Analytics Systems	26MML0006	438
5.	V. Multi-Cluster Spark Analytics Architecture	26MML0015	782
6.	VI. Distributed Data Engineering Layer	26MML0006	745
7.	VII. Spark Processing and Execution Layer	26MML0006	870
8.	VIII. Fault Detection and Monitoring Framework	26MML0054	728
9.	IX. Fault-Tolerant Recovery and Resilience Layer	26MML0015	877
10.	X. Multi-Cluster Coordination and Orchestration Layer	26MML0015	727
11.	XI. Explainability and Reliability Assessment Layer	26MML0054	722
12.	XII. Security, Governance, and Compliance Layer	26MML0047	587
13.	XIII. Experimental Setup	26MML0006	519
14.	XIV. Evaluation Metrics	26MML0054	367
15.	XV. Results and Findings	26MML0054	955
16.	XVI. Comparative Analysis	26MML0006	609
17.	XVII. Reliability and Business Continuity Assessment	26MML0047	510
18.	XVIII. Industry Use Cases and Case Study Evaluation	26MML0047	580
19.	XIX. Business Impact Assessment	26MML0054	396
20.	XX. Challenges and Limitations	26MML0015	433
21.	XXI. Future Research Directions	26MML0054	436
22.	XXII. Conclusion	26MML0015	374
	References (127)	All members	not counted
	Appendix and Data Availability	26MML0006	255

Consolidated Report on contributions		
Sl. No.	Reg. No. of Student	Total number of Words contributed across the sections
1	26MML0054	3604
2	26MML0006	3732
3	26MML0047	3383
4	26MML0015	3569
Total number of words		14288

Resilient Data Science Programming for Fault-Tolerant Distributed Analytics in Multi-Cluster Spark Environments

Dr. M Rajasekhara Babu[1]
Mudavath Narasimha Naik (26MML0054)[2], Vana Shivaramakrishna Kumar (26MML0006)[3]
Vinayak C (26MML0047)[4], K V Kartik (26MML0015)[5]

School of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore-632014, T.N., India
[1]mrajasekharababu@vit.ac.in, [2]mudavath.narasimha2026@vitstudent.ac.in, [3]vana.2026@vitstudent.ac.in, [4]vinayak.c2026@vitstudent.ac.in, [5]kartik.kv2026@vitstudent.ac.in

Corresponding author: M Rajasekhara Babu (e-mail: mrajasekharababu@vit.ac.in).

ABSTRACT Although Apache Spark can recover from the failure of a task or an executor, a job is lost if the entire cluster fails. We have developed RMC-Spark, a resilience layer that spans multiple independent Spark clusters and transfers a failed data science pipeline to a healthy cluster, resuming it from the last stage that was committed. A health monitor evaluates each cluster based on five different signals. An adaptive policy checks a stage once the amount of unsaved work reaches the Young and Daly interval, employing a running estimate of the mean time between failures. The controller only moves a job when it has stopped making progress, selects the target according to a cost function, and records every decision together with the reason for it. A small Python library converts PySpark functions into idempotent stages that are committed to a shared object store. The system was built using Docker, Spark 3.5, MinIO and Prometheus, and includes 56 automated tests to verify its logic. In 36 trials carried out on a two-cluster testbed, RMC-Spark detected each complete cluster outage in around 8 seconds, moved the job and successfully completed both the outage scenarios, whereas the single-cluster configurations failed all five of them. In the fault-free runs there was no measurable overhead, compared to 24% for fixed checkpointing, since the jobs were too short for the adaptive policy to trigger a checkpoint.

INDEX TERMS Apache Spark, Chaos Engineering, Checkpointing, Cloud Analytics, Data Engineering, Distributed Analytics, Explainable Operations, Fault Tolerance, High Availability, Multi-Cluster Computing, Resilient Data Science.

EXECUTIVE SUMMARY

The fraud checks and the production-line alarms can't afford to have a rerun [1][2], and Spark is normally the system used for this kind of work [3][4]. Although Spark is fault tolerant, this tolerance stops at the cluster boundary: if the master node fails or a zone is lost, all the jobs on that cluster are brought to a halt [5][6]. The rds library that we have developed enables the various stages in a pipeline to be resumed, and our controller transfers any stalled jobs to a healthy cluster and gives an explanation as to why. Table 1 shows the contributions and their current status.

TABLE 1
EXECUTIVE SUMMARY OF CONTRIBUTIONS AND OUTCOMES

Contribution	What it provides	Status / outcome
C1 Health monitoring	Health score H from five weighted signals with EWMA z-score anomaly detection (Section VIII)	Implemented; thresholds and weights fixed in configuration; unit tested
C2 Adaptive checkpointing	Young/Daly interval driven by an online MTBF estimate; commit-marker protocol on shared storage (Section IX)	Implemented in the rds library; formulas unit tested
C3 Cross-cluster failover	Health state machine plus progress-aware migration; restores committed stages on the target cluster (Section IX)	Implemented; simulated outage fails over within 16 s versus at least 30 s for a timeout baseline
C4 Workload orchestration	Placement cost combining load, health and data transfer (Section X)	Implemented; deterministic tie-breaking tested
C5 Explainable reliability	Decision log with per-signal contributions and a reliability score R (Section XI)	Implemented; contributions sum to R by construction
Evaluation	36-run matrix (4 configurations, 3 fault settings, 3 repetitions) and two ML case studies (Sections XIII-XV)	Cluster outage: P completed 2/2 valid runs (detection 7.6 s), B1/B2 0/5 (Fisher p = 0.048); fault-free overhead 0 % vs 24 % for fixed checkpointing
Open artefacts	Source code, configurations, datasets scripts, raw logs	Repository and data-availability statement provided

The advantage is that an outage that currently results in a failed job and the need for a manual rerun is now handled automatically through an explained recovery. In our tests, RMC-Spark managed to detect every outage within about 8 seconds and completed the job on the healthy cluster, whereas the single-cluster configurations lost all such jobs by the deadline; runs that encountered no faults were just as fast as ordinary Spark. Since our jobs were short, the adaptive policy never produced a checkpoint; it will still be necessary to measure the resumption of long pipelines (Section XXI).

I. INTRODUCTION

A. EVOLUTION OF DISTRIBUTED ANALYTICS

By combining a simple functional model with automatic partitioning and the re-execution of failed tasks, MapReduce transferred large-scale analytics to commodity clusters [7]. YARN separated resource management from the processing aspect so that different frameworks could share the same cluster [8]. Resilient distributed datasets (RDDs) kept the data in memory and substituted replication with lineage, which is the record of the transformations used to create each partition, thereby allowing lost partitions to be recomputed rather than copied [5]. Spark developed support for batch processing, SQL, streaming, and machine learning based on that abstraction [3][9][10], while Flink adopted a streaming-first approach together with distributed snapshots [11]. Figure 1 shows this history.

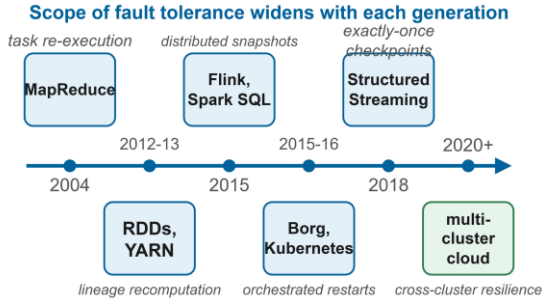


FIGURE 1. Evolution of distributed analytics platforms; each generation widened the scope of fault tolerance.

Benchmarks have consistently shown Spark to outperform disk-based MapReduce when it comes to iterative and interactive tasks, which is the reason why data scientists choose to use it [4][12]. Analytics has also shifted to the cloud: orchestrators derived from Borg and Kubernetes itself are now used to run analytics alongside services [13][14], serverless platforms have eliminated the need for cluster management [15][16], and lakes and lakehouses have separated storage from compute [17][18]. Stream processors now provide exactly-once results for unbounded input [19][20][21].

B. THE RESILIENT DATA SCIENCE ECOSYSTEM

At present, one of the analytical results relies on object storage, metadata services, cluster managers, executors, and the code itself, each of which can fail independently. Figure 2 illustrates the various people involved: data engineers are responsible for creating the ingestion and transformation pipelines, data scientists develop features and models based on these pipelines, and platform engineers ensure that the clusters continue to operate. The impact of the same failure is perceived very differently by each group. To a platform engineer a failed worker is just something they shrug off; but a data scientist who has had to restart a three-hour training run has lost an entire afternoon. Resilience must therefore be built not only into the platform but also into the programs [6][22].

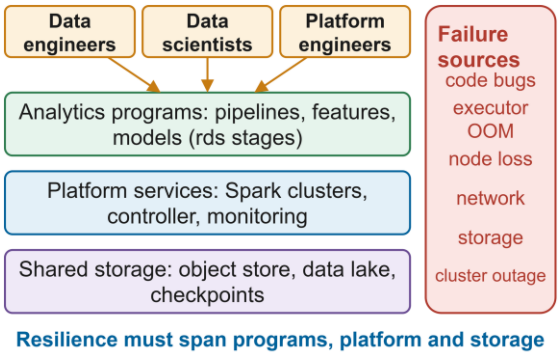


FIGURE 2. Programs, platform services, shared storage and the people around them; resilience must cover every layer.

C. FAULT TOLERANCE REQUIREMENTS OF MODERN ANALYTICS

The situation is made more difficult by three factors. The pipelines are now longer, and restoring a lineage involving dozens of stages may take longer than the initial run [23]. Gray failures, in which a deteriorating component still passes its health check, account for a large part of the unavailability experienced in cloud environments [24]. Moreover, organisations operate several independently managed clusters in different zones, clouds or business units [25][26]; since each cluster is operated separately, the jobs of a failed cluster cannot make use of the available capacity nearby.

D. MOTIVATION AND CONTRIBUTIONS

Existing solutions are either confined to a single cluster, for example lineage checkpointing [23] and write-ahead lineage [27], or else they involve restarting a black-box job from the beginning using a general high-availability script. Although a recent multi-cloud Spark scheduler incorporating micro-checkpointing fills the gap, it does so by relying on a policy that is difficult to explain [28]. This difficulty is in part organizational since analytics teams assess success according to whether reports and models are delivered on time, while platform teams look at infrastructure availability, and the two can differ: even if the platform is ninety-nine per cent available it can still fail to meet a critical deadline because of a single badly timed outage [29]. The term 'resilient data science programming' refers to writing the code and operating the platform in such a way that an outage of this kind results in only a few minutes of automatic recovery, rather than hours of reruns

We present five contributions. First, we regard resilient data science programming as consisting of resumable stages together with a cross-cluster

controller. Second, we have developed RMC-Spark, incorporating a health monitor (C1), Young and Daly adaptive checkpointing (C2), progress-aware failover (C3), cost-based placement (C4) and an explainable reliability score (C5). Third, we offer rds, a PySpark library which ensures that stages are both idempotent and resumable by means of commit markers. Fourth, we define a repeatable evaluation involving nine fault types, four workloads and five configurations, and make available the testbed. Fifth, we examine the requirements, the security aspects, the issue of continuity and industrial use. Sections II to IV deal with the problem, the related work and the requirements; Sections V to XII describe the architecture and the various layers; Sections XIII to XVII cover the setup and the results; and Sections XVIII to XXII discuss the use cases, the impact, the limitations and the conclusions.

II. PROBLEM STATEMENT

A job in a multi-cluster environment can fail at a number of different levels, as shown in Fig. 3. At the task level, an executor either crashes because of memory exhaustion or due to a bug, after which Spark just reexecutes the task [5]. At the node level, the worker process dies along with its executors and the cached partitions. Since node and disk failures happen frequently enough in production cloud environments, it is worthwhile having dedicated predictors [30][31][32][33]. It is also possible for a master, a network connection, or a zone to disappear and thereby halt all the jobs on the cluster, and grey failures such as network delay, a slow disk, or a CPU that has been starved gradually slow down the work without any error being raised [24].

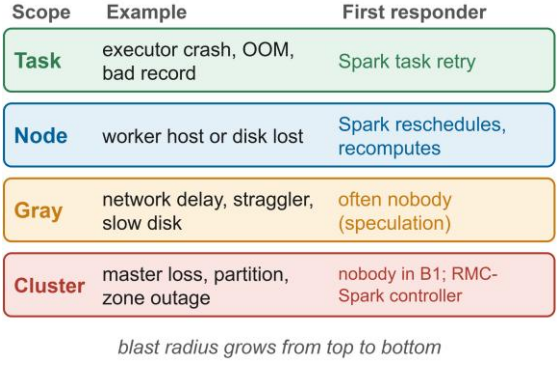


FIGURE 3. Failure scenarios ordered by blast radius, with the first responder in standard Spark.

They lead to five practical problems. When a job is interrupted under standard Spark it has to be resubmitted manually. The problem of stragglers causes an imbalance which speculation only partly corrects [34]. Data that is stored only in lost executors results in a long recomputation [23]. Both recomputation and gray failures lead to a loss of performance. Moreover, downtime accumulates as operators investigate the incident and restart the jobs, even though faults are still frequent in large data centres [35].

Job J is formally composed of a series of stages s1 through sn which are processed on one of K clusters, with computing times c1 to cn respectively, and cluster k fails according to a renewal process having a mean time between failures of Mk. Our objective is to maximise the probability that job J satisfies its deadline, to minimise the expected completion time, to keep the fault-free overhead low, and to ensure that all recovery actions are based on evidence that an operator can examine. The research question therefore concerns how a pipeline running on one of several independent Spark clusters can continue operation after the cluster it is using fails, with a bounded amount of lost work, without unnecessary restarts and with decisions that can be audited. Table 2 relates this question to existing gaps in the literature: the surveys identify proactive, cross-layer and explainable methods as open problems [6][36][37], and there is a need to enforce availability targets at runtime [29][38].

TABLE 2
RESEARCH CHALLENGES AND GAPS

Challenge	State of practice	Gap addressed here
Cluster outage	Job fails; manual or scripted restart from scratch	Automatic failover with stage-level resume
Long lineage	Recompute from source or checkpoint everything	Checkpoint interval derived from observed failure rate
Gray failures	Usually invisible to health checks	Latency and progress signals in the health score
Unnecessary restarts	Timeout-based HA restarts healthy jobs	Migrate only jobs that stopped progressing
Opaque decisions	Learned or scripted policies without rationale	Per-signal contributions logged for every decision
Reproducible evaluation	Ad hoc fault experiments	Scripted fault injection and statistical comparison

III. BACKGROUND AND LITERATURE REVIEW

We look at the related work according to the taxonomy shown in Fig. 4: that concerning recovery within the engine, checkpoint intervals, stragglers and gray failures, failure prediction and AIOps, multi-cluster orchestration, and resilience engineering. Table 3 gives a comparison of the various approaches.

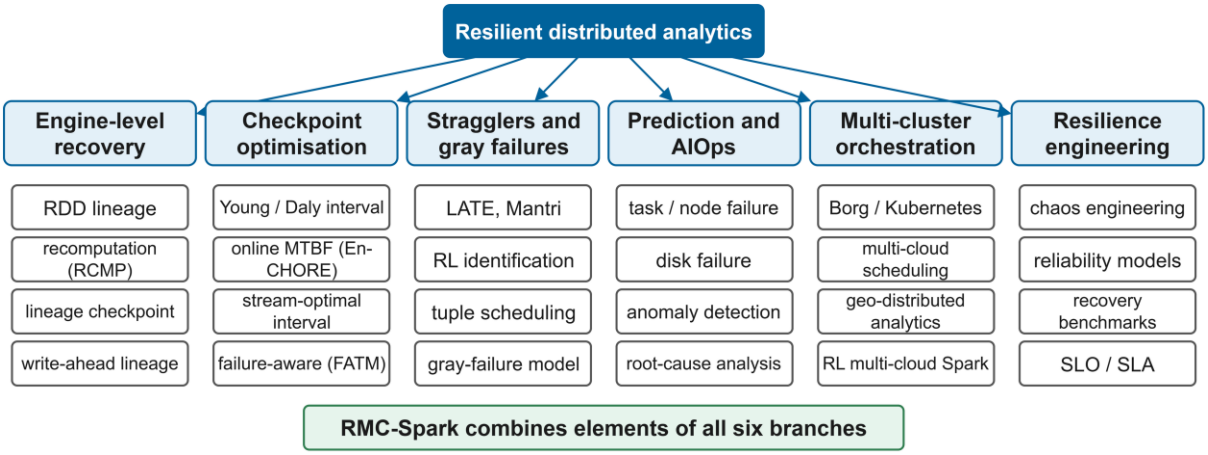


FIGURE 4. Taxonomy of the reviewed literature; RMC-Spark combines elements of all six branches.

A. FAULT TOLERANCE INSIDE THE SPARK ENGINE

Spark recovers mainly through lineage: a lost partition is rebuilt by rerunning the deterministic transformations that made it, and long lineage can be checkpointed to reliable storage [5]. Spark SQL and Structured Streaming restart from query plans and write-ahead offset logs [9][10]. Surveys credit lineage for much of Spark's success, but they also note that recomputation becomes expensive for iterative and long-running applications, where lineage can stretch across many shuffles [4].

The RCMP contend that selective persistence together with recomputation can take the place of replication [39]. Kweun et al. checkpointer the lineage graph rather than the data [23], and write-ahead lineage allows pipelined engines to recover tasks without carrying out a global rollback [27]. Other research focuses on partition balancing by means of checkpoint-based cache recovery [40], erasure-coded caching [41] and GraphX iterations [42]. A systematic survey identifies adaptive, cross-layer designs as still being open problems [6]. All of these methods assume that the cluster remains operational. None of them are capable of moving a running job to a different cluster when it does not.

B. CHECKPOINT INTERVAL OPTIMISATION

The square root of twice the checkpoint cost multiplied by the MTBF is Young's first-order optimum interval [43], and Daly's higher-order terms remain accurate even when the cost is not small [44]. Later work deals with an MTBF that varies with time; Sigdel et al. count the failures that occur while a job is running, estimate the MTBF as they go and omit checkpoints that would not be worthwhile [45]. With respect to streams, Jayasekara et al. derived an interval that is optimal in terms of utilisation [46][47], similar analyses are available for Flink [48], and FATM checkpoints are based on the observed failure rate [49]. Incremental checkpointing reduces the per-checkpoint cost [50], and convergence-aware policies are aimed at training jobs [51]. We apply the Young and Daly interval to entire stages of the data science pipeline, and the MTBF used in the interval is kept current by the cross-cluster controller based on the failures it actually observes.

C. FAULT RECOVERY BENCHMARKS FOR ANALYTICS FRAMEWORKS

Van Dongen and Van den Poel carried out measurements of outage, downtime, recovery time, data loss, and the number of duplicates for four stream engines under various kinds of failure [52]. Vogel et al. carried out the same study on Kubernetes using chaos-engineering fault injection; in their experiment Flink proved to be the most stable and Structured Streaming recovered reliably although with greater latency [53]. PGVal includes reliability and failure-cost metrics [54], and a survey examines stream fault tolerance from replication to snapshots [19]. All of these benchmark tests involve a single cluster. We take over their metric definitions, particularly that based on throughput for recovery, and also include the scenario in which the entire cluster is lost.

D. STRAGGLERS AND GRAY FAILURES

Engineers have been concerned about stragglers since the very first MapReduce clusters. LATE enhanced speculative execution on mixed hardware [55] and Mantri took action with regard to outliers [56]. Subsequent approaches have learned to detect stragglers using reinforcement learning [57] or neural networks [58]; a survey addresses mitigation techniques for Hadoop [34] and Hone schedules tuples in order to prevent stream stragglers [59]. Gray failures are more extensive in scope: Huang et al. refer to them as differential observability, meaning that a component is seen as healthy by one observer but as failed by another [24]. RMC-Spark detects these failures by combining master reachability, probe latency and driver-reported progress.

E. FAILURE PREDICTION AND AIOPS

A great deal of effort is directed towards predicting failures before they occur. Bi-LSTM models predict task and job failures based on traces [35], predictors have been compared using Google traces [60], and the failures of cloud ML jobs have been characterised [61]. Proactive schedulers combine prediction with remediation [62], online VM failure

prediction helps to ensure high availability [63], and prediction of node and disk failures is carried out on a large scale [30][31][32][33], while workload prediction assists with capacity planning [64].

The surveys on AIOps categorise failure management according to intervention time [65] and connect it with MLOps [66] and language models [67]. Anomaly detection based on metrics and logs has been the subject of numerous surveys [68][69][70][71], covering monitoring data for use in cloud detection [72] and for Kubernetes [73]. Root cause analysis varies from graph ranking [74] to log-based explanations [75], with surveys addressing both of these approaches [76][77]. The same point is repeatedly made in this body of work: operators only act on outputs that they are able to interpret, as Li et al. observed when implementing failure prediction on an extremely large platform [30]. That is the reason why RMC-Spark employs a transparent additive health model instead of a black-box predictor.

F. ORCHESTRATION ACROSS CLUSTERS AND CLOUDS

Borg introduced fault-tolerant large-scale scheduling [13] and played a key role in shaping Kubernetes [14], the scheduling aspects of which have been extensively studied [78][79][80]. Cost-aware and simulation-driven orchestration has been suggested [81][82], containers have now reached the field of high-performance computing [83], multi-cloud frameworks have been reviewed [25], and schedulers take into account security and reliability constraints [84]. Geo-distributed analytics is used to optimise both data placement and data transfer [85][86][87][88][89], as outlined in [90]. The work most similar to ours is that of Raj et al., who for multi-cloud Spark workflows combine a reinforcement-learning scheduler with predictive provisioning and micro-checkpointing [28]. We agree with their objective but employ an analytical policy, an explainable placement cost, and real fault injection.

G. CLOUD FAULT TOLERANCE AND RESILIENCE ENGINEERING

The work on cloud reliability includes proactive tolerance in federations [91], risk-driven actions [92], service reliability models [93] and availability models for edge, fog and cloud [94], together with several surveys [22][36][37]. Chaos engineering was started at Netflix [95] and has since been applied to security [96], containers [97] and digital twins [98], with a recent review of the practice [99]. Studies on Spark tuning [100][101][102][103], performance prediction [104], interference diagnosis [105] and engine comparisons [106] demonstrate how environment-sensitive analytics is.

H. DATA PLATFORMS AND PIPELINE RELIABILITY

Research into data lakes includes studies of their architecture and metadata [17][107], and lakehouses incorporate transactions onto object storage [18][108], something that is important for recovery since a resuming job needs to be able to distinguish between committed results and partial ones. A great deal of pipeline failures are due to data rather than to infrastructure [109], quality frameworks include checks at various points along the pipeline [110], and schema evolution poses a threat to the stored intermediates [111]. Durable state is stored in object storage and is protected by a lightweight commit protocol rather than by a full table format, thereby keeping the number of dependencies low while at the same time making sure that no partially written stage is ever read.

I. SYNTHESIS

What stands out are the following three points: although there are well-established methods for reducing the cost of recovery within a single engine and checkpoint theory is settled, the two are rarely combined with actual information on how often the platform fails. While failure prediction and anomaly detection are areas of active research, the results of this research seldom lead to a specific and clearly explained recovery action for an analytics job. Most research involving multiple clusters focuses on optimising placement and scheduling, not on how to continue a job after the cluster it is running on has disappeared. RMC-Spark is designed to address the gap left by these three points.

J. COMPARATIVE ANALYSIS AND POSITIONING

Table 3 evaluates the various approaches with respect to the five properties which Section II requires: recovery after the loss of an entire cluster, adaptive checkpointing, awareness of gray failures, explainable decisions and the ability to carry out a repeatable fault-injection evaluation. No method possesses all five of these properties. Those at the engine level are efficient but are restricted to a single cluster, AIOps methods are able to detect problems but seldom take action on analytics jobs, and multi-cloud schedulers operate across clusters but provide little explanation. RMC-Spark is intended as a combination of these features, at the cost of the overheads and the single-host limitations mentioned in Section XX.

TABLE 3
COMPARATIVE ANALYSIS OF EXISTING APPROACHES

Approach	Main idea	Cross-cluster recovery	Adaptive checkpoint	Gray-failure aware	Explainable	Fault injection	Analytics engine
RDD lineage [5]	Recompute lost partitions	No	No	No	n/a	No	Spark
RCMP [39]	Recomputation instead of replication	No	Partial	No	n/a	Yes	Hadoop
Lineage checkpoint [23]	Checkpoint lineage graph	No	No	No	n/a	Yes	Spark
Write-ahead lineage [27]	Log lineage at runtime	No	No	No	n/a	Yes	Quokka
En-CHORE [45]	Online MTBF, skip checkpoints	No	Yes	No	n/a	Simulated	Generic
Optimal stream checkpoint [47]	Utilisation-optimal interval	No	Yes	No	n/a	Yes	Flink
FATM [49]	Failure-rate-driven checkpoints	No	Yes	No	n/a	Yes	Stream
Hawkeye [57]	RL straggler identification	No	No	Yes	Low	No	Spark
Bi-LSTM prediction [35]	Predict task/job failures	No	No	Partial	Low	Trace	Cloud
Proactive scheduling [62]	Predict and remediate	No	No	Partial	Low	Trace	Cloud
Recovery benchmark [53]	Chaos-based comparison	No	No	No	n/a	Yes	Flink, Spark, Kafka
Multi-cloud RL scheduler [28]	RL scheduling, micro-checkpoints	Yes	Yes	Partial	Low	Not stated	Spark
RMC-Spark (this work)	Health-driven failover, resumable stages	Yes	Yes	Yes	Yes	Yes	Spark

Our assessment was also influenced by the review. Since published recovery benchmarks only cover one cluster, we have included the scenario involving a lost cluster and have compared the various recovery strategies together on a single engine. Although adaptive checkpointing has mostly been studied in the context of stream operators and HPC jobs, Section XV discusses its behaviour in the case of short data science pipelines, including the instance where it chooses not to checkpoint at all.

IV. REQUIREMENTS ANALYSIS FOR RESILIENT ANALYTICS SYSTEMS

The requirements are derived from the problem statement as well as from the gaps in Table 2 and from service-level practice [29][38]; Figure 5 links the stakeholders' concerns to the requirements, the components and the verification.

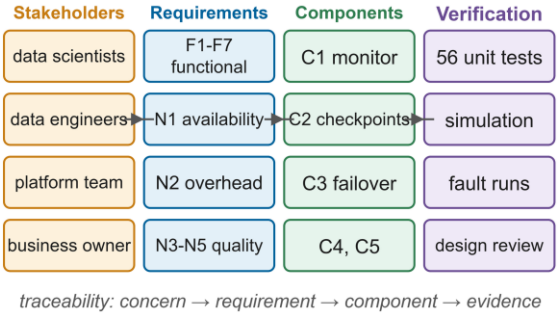


FIGURE 5. Stakeholder concerns refined into requirements, traced to components C1-C5 and to their verification.

What scalability entails is that when a cluster or a workload is added, the controller's design remains the same, the cluster registry consisting of plain data and the control loop increasing linearly as the number of clusters and jobs does. For availability, a job must be able to carry on if its cluster fails provided that another cluster is still working. Recoverability is achieved by setting bounds on the amount of lost work through the use of the checkpoint policy. Reliability requires that recovery never causes any harm on its own, for instance by restarting a job which is still making progress. Interoperability means that existing PySpark code will only need a small amount of modification, which is the reason why we have developed a decorator library rather than introducing a new language. Governance means that every automatic decision is logged together with its reason and that data movement conforms to the policy [112][113]. Finally, continuity implies graceful degradation in that jobs will wait and then resume rather than fail when no cluster is healthy, and data quality [109][111] means that half-written outputs are never read.

A. SCALABILITY AND AVAILABILITY REQUIREMENTS

We look at scalability in terms of three factors: the number of clusters, the number of jobs running at the same time, and the size of the data. Since each poll involves one HTTP request for each cluster and two requests for each running driver, the amount of data affects only the cost of a checkpoint, a factor that is already taken into account by the adaptive policy. Availability is defined on a per-job basis: if a job is submitted when at least one cluster is healthy, it should still complete even if the cluster that it uses fails, a standard that is more rigorous than simple infrastructure availability since the job's progress also has to be preserved.

B. RECOVERABILITY AND RELIABILITY REQUIREMENTS

Recoverability can only be measured in two ways. Firstly, a cluster failure has to be detected and dealt with within a number of polling intervals together with a brief grace period to allow progress to be made, and secondly, the work carried out after recovery must be limited by the checkpoint interval and not by the length of the job. Reliability includes three conditions: it must not restart a job which would have completed on its own, a single slow probe must not result in a failover, and a recovered cluster must not oscillate between states.

C. INTEROPERABILITY, GOVERNANCE AND CONTINUITY REQUIREMENTS

The method should be based on standard Spark, storage interfaces and PySpark; actions have to be traceable and auditable; and work that has been

TABLE 4
FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS MATRIX

ID	Requirement	Component	Verification
F1	Probe every cluster and running driver periodically and compute a health score	C1 monitor, health model	Unit tests; Fig. 23 time series
F2	Detect a failed cluster within a few polling intervals without reacting to single glitches	C1 state machine	Unit tests (consecutive-poll rule)
F3	Checkpoint pipeline stages adaptively and commit them atomically	C2 rds library	Unit tests; checkpoint metrics
F4	Resume a job on another cluster from its last committed stage	C2 + C3	Cluster-outage experiments
F5	Migrate only jobs that stopped progressing	C3 jobs manager	Simulation test; master-kill experiments
F6	Place jobs on the best available cluster	C4 orchestrator	Unit tests
F7	Record every decision with its cause and expose reliability scores	C5 decision log	Unit tests; API
N1	Availability: no job loss when one cluster fails and another is healthy	Whole system	Success rate under cluster faults
N2	Overhead: limited slowdown in fault-free runs	C2 policy	Clean-run overhead, Eq. (11)
N3	Reproducibility: every experiment scripted and seeded	Harness	Repository, configuration files
N4	Interoperability: unchanged Spark, standard PySpark APIs	rds library	Workloads use stock Spark 3.5
N5	Security and governance: auditable actions, least privilege in production	Section XII	Design review

D. REQUIREMENT PRIORITISATION

In the MoSCoW system, F1 through F5 and N1 are classified as musts since, in the absence of these, a job could not survive beyond the lifetime of its cluster; likewise, N3 and N4 are also musts when dealing with a research prototype. F6, F7 and N2 are considered shoulds as they contribute to efficient behaviour and auditability, and the production controls associated with N5 are merely coulds in the case of the testbed, which is partly built and fully designed. The experiments provide direct verification of N1 and N2, while the automated tests take care of the remainder.

V. MULTI-CLUSTER SPARK ANALYTICS ARCHITECTURE

A. DESIGN PRINCIPLES

Four The design was based on four principles. The first was to leave the engine unchanged: each cluster operates using the standard Apache Spark in standalone mode, meaning that this method is compatible with current setups. The second principle was to separate jobs from clusters: a job has a permanent identifier, its persistent state is stored in shared object storage, and the cluster that is carrying out an attempt can be replaced if necessary. The third principle was to react only when harm has been proven, not when there is only suspicion: a cluster that is not working properly is not given any new tasks, but a job only moves when it has stopped making progress. The fourth principle was to explain all actions, since operators will not trust a recovery process that they cannot verify [30][65].

B. OVERALL ARCHITECTURE

Figure 6 illustrates the architecture that has been deployed. Each of the two Spark clusters, named A and B, consists of one master node and two workers. A common MinIO object store, accessed via the Hadoop S3A connector, is used for storing the inputs, the stage and streaming checkpoints, and the results. Metrics are collected by Prometheus and are displayed using Grafana. The controller makes use of its own container and within it hosts the Spark drivers, starting up each job by means of spark-submit in client mode, which ensures that each driver runs outside of the cluster whose executors it employs. A fault injector acts on the containers of each cluster by means of the Docker Engine API.

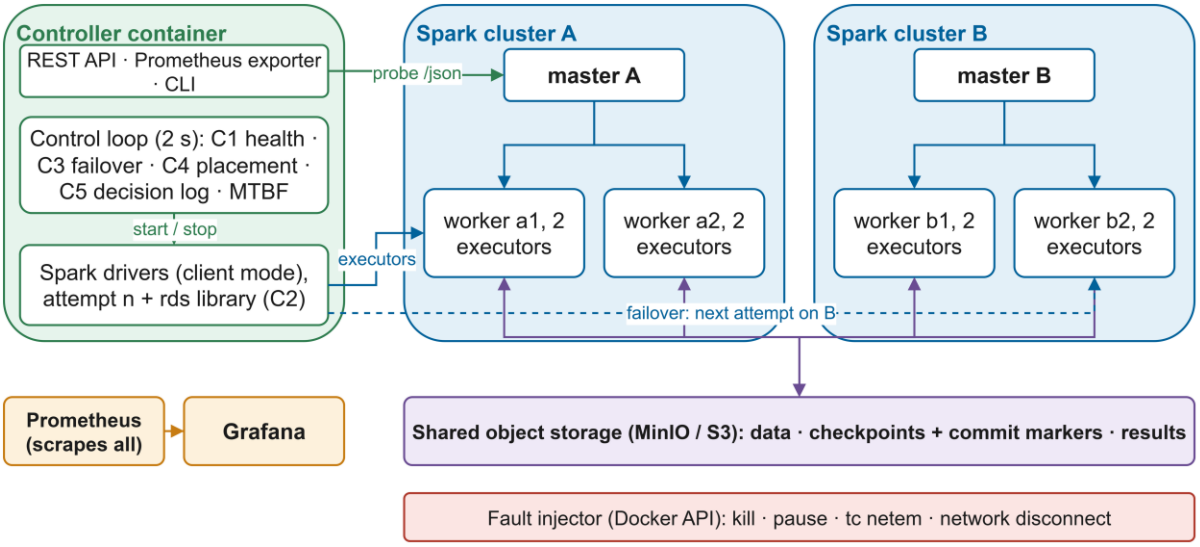


FIGURE 6. Resilient multi-cluster Spark architecture (core figure): drivers run in the controller, executors on cluster A or B, and durable state in shared storage, so a job can resume on either cluster.

The choice of whether to put the drivers into the controller is the one that determines all the other decisions. When running in cluster mode the driver operates on a worker and dies along with the cluster; in client mode it outlives the cluster and the controller is then able to stop it and start a new one on another master. The controller also detects failures from two perspectives, that is, from the master's point of view regarding the cluster and from the driver's point of view concerning the progress of the task, so that it can distinguish between a master that is dead but has working executors and a dead cluster, a distinction which is required by gray failures [24].

C. COMPONENTS AND INTERACTIONS

Every two seconds the controller checks the JSON endpoint of each master and the REST API of each running driver, calculates a health score for each cluster (C1), and updates the failure-detector state machine for that cluster. It then applies the recovery rules to each job (C3) and, if a job needs to start or be moved, selects a cluster based on placement cost (C4). Each state change and action is recorded together with the contribution of each health signal (C5). The data science programs make use of the rds library, which offers resumable stages and an adaptive checkpoint policy (C2); the controller estimates the mean time between failures from the observed outages and passes this estimate to each attempt.

D. LAYERED ANALYTICS STACK

The system is depicted in Figure 7 in the form of layers, a structure that is then continued in Sections VI to XII. Located at the bottom is the shared object storage, with the Spark clusters and the rds library situated inside each driver above it. The control layer is responsible for monitoring, detection, recovery and orchestration, the explainability layer converts the decisions into auditable records and scores, and governance applies to all the other layers. Observability is incorporated into every layer, as is typical of container-based systems [114][115].

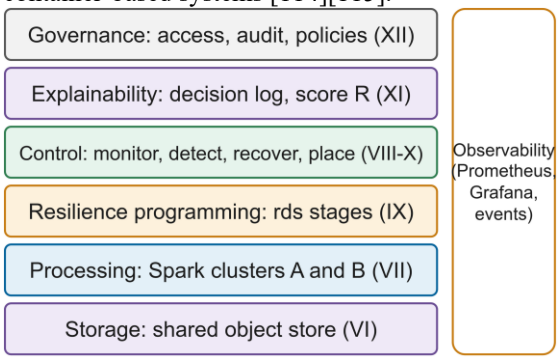


FIGURE 7. Layered analytics stack; observability and governance span all layers, each detailed in Sections VI to XII.

E. FAILURE DOMAINS AND ASSUMPTIONS

We identify three failure domains. So long as one cluster domain remains, the system can tolerate the loss of any number of cluster domains (that is, one master and its workers). It is assumed that the storage domain is available, and in a production environment this means that the service is replicated, while the control domain—comprising the controller and its drivers—runs using replicas. Failures are of the fail-stop or fail-slow type; Byzantine behaviour is not included. It is assumed that the stage functions are deterministic, since Spark's lineage recovery makes this assumption [5].

F. DEPLOYMENT MODEL

The All of the processes are managed by Docker Compose on a single machine, ensuring that the experiments can be repeated and any faults can be accurately identified. In a production environment, A and B would each be separate deployments located in different zones or clouds, together with a replicated object store and redundant controllers. Since multi-cloud orchestration and geo-distributed analytics are already capable of handling data placement between these locations [25][90], there is a data-transfer term in the placement cost discussed in Section X, whereas in our testbed with shared storage the term is zero.

G. ARCHITECTURE IN THE EXECUTED EXPERIMENTS

The experiments employed exactly that architecture: each job began in client mode directed at master A, each failure affected the job's current cluster, and the same control loop which handles the API was the one that carried out all the recoveries. During the.

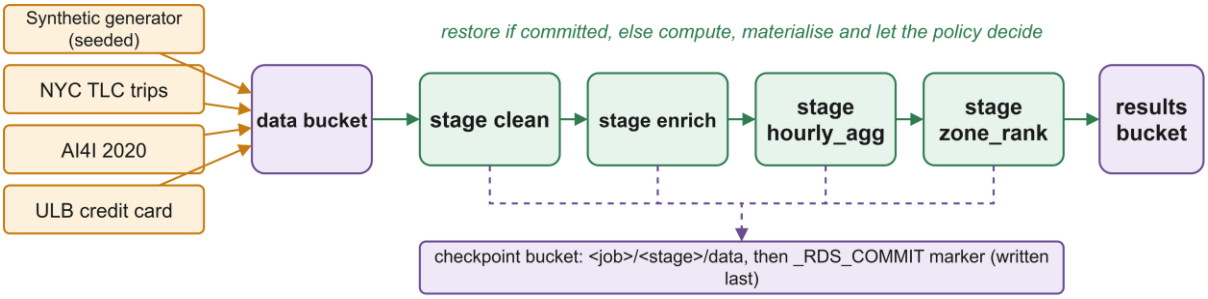


FIGURE 8. Data engineering pipeline: inputs in the data bucket, resilient stages committing to the checkpoint bucket, outputs in the results bucket.

A seeded generator creates four million synthetic taxi trips with about one per cent deliberately invalid for the cleaning stage, and a skewed graph with two million edges for the iterative workload, so every team member and reviewer has the same data. A download script and ingestion job adds the NYC taxi records, the AI4I 2020 maintenance data and the ULB credit-card fraud data, and a normalisation function maps the real taxi schema onto the synthetic one.

VI. DISTRIBUTED DATA ENGINEERING LAYER

A. INGESTION AND STORAGE ORGANISATION

All durable data resides in an S3-compatible store with three buckets: data for inputs, checkpoints for stage and streaming checkpoints, and results for outputs. Object storage isolates data from compute, as do data lakes and lakehouses [17][18][107][108], and it enables cross-cluster recovery as any cluster can read any committed stage. The Fig. 8 shows the pipeline the workloads use.

B. TRANSFORMATION WITH RESILIENT STAGES

Transformations are PySpark functions decorated as stages of a ResilientPipeline. In the ETL workload a cleaning stage drops trips with implausible duration, distance, fare or speed, an enrichment stage joins zones with a borough lookup, an hourly aggregation computes counts, mean and 95th-percentile fares, speeds and tip rates, and a ranking stage uses a window function to pick each borough's top revenue zones. Each stage is a pure function of its inputs so rerunning it and reloading its output is equivalent.

If If an earlier attempt left a commit marker, the stage's output is read back and the function skipped. Otherwise the function runs, its result is persisted and forced with a count so the recorded compute time is genuine, and the checkpoint policy decides whether to write it. Data is written first and a small JSON marker second, so a stage counts as committed only when complete, and half-written output from a crashed attempt is never read. recomputation, and the next attempt simply overwrites any partial output it finds.

C. STREAMING DATA PATH

Streaming proceeds in micro-batches rather than stages. Structured Streaming already saves offsets and state after each micro-batch [10]; RMC-Spark puts that location in the shared checkpoint bucket so a query resubmitted on another cluster under the same identifier continues from its last committed batch. For each micro-batch the driver logs one event with the input row count, the processing rate and the batch duration, which the analysis uses to measure how quickly throughput recovers after a fault.

.

D. METADATA MANAGEMENT AND SCHEMA EVOLUTION

Each marker records the stage name, its position, job identifier, attempt number and timestamp. Together the markers form the resilience layer's metadata catalogue read at the start of each attempt. In strictly linear pipelines (the iterations of an iterative algorithm) the library skips every stage before the latest committed one rather than recompute results that a later checkpoint contains. Stage timings go into a per-job event log used to compute recomputed work and checkpoint cost.

Stage outputs are Parquet files that carry their own schema. Since the stage identity includes the job identifier, changed pipeline code should have a new identifier so old checkpoints are never mixed with new code. Schema evolution techniques [111] and automated governance [113] could extend this, for example by storing a code version in the marker

E. PARTITIONING AND STORAGE MANAGEMENT

We use 16 shuffle partitions with adaptive query execution, and inputs are generated in 16 partitions for the four executor cores per job. Checkpoint data is kept per job identifier so storage grows with the number of jobs and can be reclaimed by a retention policy once a job has succeeded. Table 5 summarises the components. Unreliable intermediate data and missing validation are common causes of pipeline failure [109][110]; the commit protocol covers reliability and validation remains the pipeline's job.

TABLE 5
DATA ENGINEERING COMPONENTS AND RESPONSIBILITIES

Component	Responsibility
Data bucket	Immutable inputs (synthetic and public datasets)
Checkpoint bucket	Committed stage outputs, commit markers, streaming offsets and state
Results bucket	Final outputs and JSON job summaries
Data generator	Seeded synthetic trips and skewed graph
Ingestion job	Copies downloaded public datasets into the data bucket
ResilientPipeline	Stage restore, materialisation, policy decision, atomic commit
Commit marker	Stage metadata: name, sequence, job, attempt, time
Event log	Per-stage timings and checkpoint decisions for analysis

F. DATA QUALITY IN RESILIENT PIPELINES

Resumption rules out reruns that read half-finished files and quietly produce wrong totals, but can reuse stale results if inputs change between attempts. A reloaded stage reflects the inputs that existed when it was computed. We assume inputs stay fixed during a job, which holds for our batch workloads; changing inputs would need an input version in the identifier or marker, follow Section XII.

G. DATA VOLUMES IN THE EXPERIMENTS

The executed matrix used 1.5 million trips in 16 partitions, giving warm fault-free ETL runs of one to two minutes. Committing every stage cost B2 four checkpoints per clean run, raising its median runtime from 92 s to 114 s, while P's adaptive policy wrote none

VII. SPARK PROCESSING AND EXECUTION LAYER

A. SPARK EXECUTION MODEL

Each cluster runs Spark 3.5.3 standalone: a master tracks workers and allocates resources, and workers start executors. Fig. 9 shows the setup. Every application starts with a driver that builds the program's logical plan, optimises it, splits it into jobs and stages, and schedules tasks on the executors [3][9]. Executors run the tasks, cache partitions, and send status and heartbeats back to the driver.

start executors. Fig. 9 shows the setup. Every application begins with a driver, which builds the program's logical plan, optimises it, splits it into jobs and stages, and schedules tasks on the executors [3][9]. Executors run the tasks, cache partitions, and send status and heartbeats back to the driver.

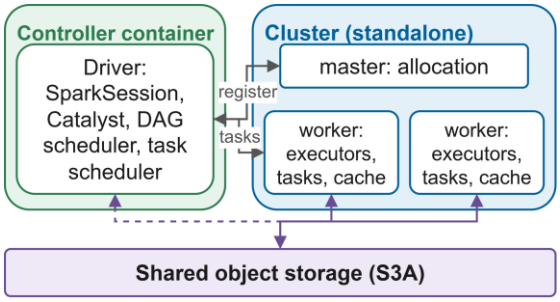


FIGURE 9. Spark execution in RMC-Spark: the driver runs in the controller in client mode and executors on the selected cluster use shared storage.

The controller starts each driver in client mode with 512 MB of memory; executors get one core and 576 MB, up to four cores per job. The driver advertises the controller's host name so executors on either cluster can reach it, and the controller assigns its UI port to read progress with Spark's REST API.

B. RDD LINEAGE AND DAG SCHEDULING

Spark represents a computation as a DAG of RDDs linked by narrow and wide dependencies [5]. A narrow dependency (filter, map) computes each partition from one parent partition, so such steps are pipelined within a single stage. A wide dependency (group-by, join that needs shuffle) reads many parent partitions and starts a new stage. The DAG scheduler submits stages in topological order and the task scheduler prefers executors that cache the input. Fig. 10 shows the ETL workload's DAG with the resilient stage boundaries.

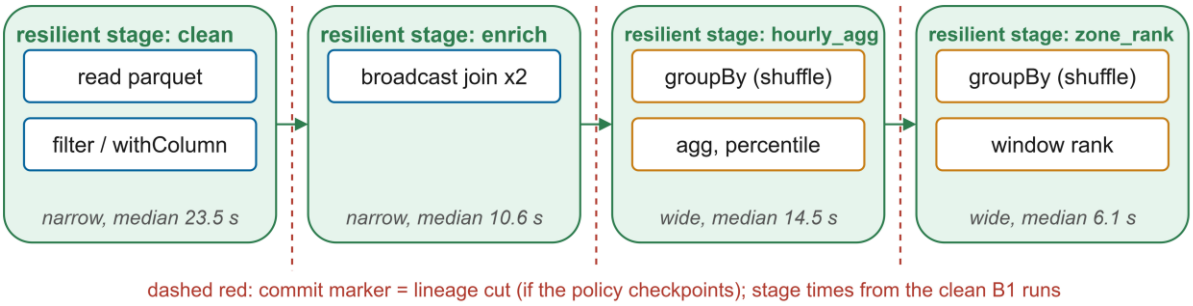


FIGURE 10. DAG of the ETL workload: Spark stages inside resilient stages; a committed stage cuts the lineage.

When an executor is lost, the driver reruns the smallest set of tasks that rebuild its partitions, sometimes reaching back through several shuffles. With many wide dependencies this is the long-lineage problem [23]. A committed resilient stage cuts the lineage: downstream stages read from object storage, and Section IX's policy decides where such cuts pay off.

C. TASK EXECUTION AND BUILT-IN FAULT TOLERANCE

Spark's own mechanisms stay on in every configuration. A failed task is retried up to four times; executors send heartbeats every 10 s and are declared lost after the 60 s network timeout; adaptive query execution tunes shuffle partitions.

The proposed configuration also enables speculative execution (Spark's standard answer to stragglers [55]). Spark tuning and performance prediction are active topics [100][104], but we fixed and documented our settings so that differences come from the resilience mechanisms, not tuning.

RESOURCE ALLOCATION AND UTILISATION

Each worker offers two cores and 1280 MB, so a cluster has four cores. The controller requests four cores per job (a whole cluster), modelling one important job with a dedicated cluster. Utilisation is measured from the cores each master reports, sampled every control-loop pass:

$$U = \frac{\sum_t c_{\text{used}}(t)\Delta t}{\sum_t c_{\text{total}}(t)\Delta t} \qquad (1)$$

where $c_{used}(t)$ and $c_{total}(t)$ are the cores used offered at sample t and Δt is the time to the next sample. Eq. (1) shows how fully a job uses its cluster and how much capacity idles during recovery. Resource management is central to Spark performance [4], and Fig. 23 in Section XV uses this measure to show load moving between clusters.

E. INTERACTION OF BUILT-IN AND EXTERNAL RECOVERY

The two recovery layers must not interfere. If Spark is still retrying when the controller considers migration, the rule that a FAILED cluster must have gone ten seconds without any finished task means Spark has had its chance and failed. A long but healthy task could trip the stall signal, so stall alone only makes a cluster SUSPECT, and migration also needs FAILED. The mechanisms are therefore stacked: Spark handles whatever it can inside the cluster, and the controller acts only when the evidence says the cluster as a whole cannot carry on.

F. DISTRIBUTED PROCESSING FLOW

Every job follows the same flow. The controller places it and launches attempt zero; the driver gets executors and the rds library reloads committed stages, computes the rest and checkpoints as the policy decides. While the driver writes stage events, the controller reads task progress from it. It records a success when the job finishes, or, if the driver crashes or the cluster fails while the job is stalled, starts the next attempt on the same or another cluster according to Section IX.

G. MEASURED EXECUTION PROFILE

In B1's clean runs the median compute time was 23.5 s for cleaning, 10.6 s for enrichment, 14.5 s for aggregation and 6.1 s for ranking, about 55 s in total, against a median completion time of 92 s. The rest went on starting the driver and acquiring executors, 13 s to 35 s per attempt, and writing results. For short jobs this fixed cost is about as large as the work itself, which is why restarting a short job on another cluster costs little more than running it normally.

Over the whole experiment, utilisation was 0.86 for cluster A and 0.04 for B. This reflects the design, since every run started on A so faults could be aimed at a known cluster; in production the placement cost would spread jobs over both clusters, so the second cluster adds throughput instead of idling.

Speculative execution, enabled in the proposed configuration, changed no fault-free result. In the executor-kill runs P finished in a median of 140 s, against 148 s for B2 and 160 s for the single valid B1 run; with one to three runs per configuration this gap is within the testbed's normal variation, and we do not credit it to speculation.

VIII. FAULT DETECTION AND MONITORING FRAMEWORK

A. MONITORING AND OBSERVABILITY ARCHITECTURE

Observability targets two audiences: operators want dashboards and the controller needs signals. Fig. 11 shows how one setup serves both. Spark masters and workers publish Prometheus metrics, Prometheus scrapes them every five seconds with the controller's own metrics, and Grafana shows health scores, states, workers, utilisation, failovers and reliability scores. Such pipelines are now standard for container-based systems [114][115]. For decisions, though, the controller probes masters and drivers directly, to keep detection free of scrape delays.

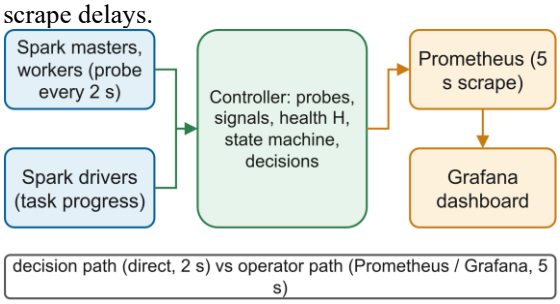


FIGURE 11. Monitoring architecture: the controller probes masters and drivers directly; Prometheus and Grafana give operators the same signals.

B. HEALTH SIGNALS

Five signals per cluster cover the failure scenarios of Fig. 3. Unreachable fires when the master's JSON endpoint does not answer within the 1.5 s timeout or reports a status other than alive. Workers-lost is the share of expected workers not listed as alive. Latency targets gray failures: we track probe response time with an exponentially weighted mean and variance, and compute the standardised deviation of Eq. (2):

$$z_t = \frac{x_t - \mu_{t-1}}{\sigma_{t-1}} \qquad (2)$$

where xt is the probe latency at poll t and μt−1, σt−1 are the EWMA mean and standard deviation before it. The anomaly level ramps linearly from zero at |z| = 3 to one at |z| = 8, and samples above the lower threshold are kept out of the baseline so a long slowdown cannot become the new normal, a known trap of adaptive detectors [68][69]. Stall is the time since the job last finished a task divided by

120 s, and executor-loss the share of the job's executors reported lost. These driver-side signals show what the job actually experiences, that the master alone cannot show in a gray failure [24].

C. HEALTH ASSESSMENT

Each signal i gives an anomaly level $a_{i,k}$ between zero and one for cluster k, combined additively:

$$H_k = 1 - \sum_{i=1}^{5} w_i a_{i,k} \quad (3)$$

with weights wi that sum to one (Table 6). Severity sets them: an unreachable master (0.45) and lost workers (0.25) mean lost capacity, a stall (0.15) lost progress, odd latency (0.10) degradation, and executor loss (0.05) is largely handled by Spark. A reachable master reporting zero workers is forced to zero health. Since the model is a sum, each contribution wi ai,k can be read directly, and the two largest are logged as the cause of every state change. Such detectors are less expressive than deep models but predictable and explainable, which AIOps adoption requires [65][72].

TABLE 6
MONITORING METRICS AND THRESHOLDS

Signal / parameter	Source and definition	Value	Effect
Poll interval	Controller control loop	2 s	Detection granularity
Probe timeout	HTTP request to master /json	1.5 s	Unreachable if exceeded
unreachable weight	Master probe failed or status not ALIVE	0.45	Anomaly 1 when unreachable
workers_lost weight	1 − alive / expected workers	0.25	Linear in fraction lost; 0 workers forces H = 0
latency weight	EWMA z-score of probe latency (α = 0.2)	0.10	Anomaly ramps from z-score magnitude 3 to 8
stall weight	Time since last completed task / 120 s	0.15	Anomaly 1 after 120 s without progress
executor_loss weight	Lost / seen executors of running job	0.05	Anomaly 1 if all executors lost
Suspect threshold	H below this value	0.95	Cluster avoided for new placements
Failed threshold	H below this value for 3 consecutive polls	0.50	Cluster declared FAILED
Recovery rule	H at least 0.95 for 5 consecutive polls	—	FAILED to HEALTHY
Migration stall	No task progress on a FAILED cluster	10 s	Job migrated
Baseline timeout (B3)	Master unreachable continuously	30 s	Naive failover

D. FAULT DETECTION WORKFLOW

Fig. 12 shows the three-state machine. A cluster is HEALTHY at a score of 0.95 or more. Below that it is SUSPECT: still eligible but its lower health raises its placement cost so new work goes elsewhere, and running jobs are untouched. It becomes FAILED only after three polls in a row below 0.50, about six seconds of strong evidence, and returns to HEALTHY only after five polls at 0.95 or more. With a 2 s poll interval, one slow probe therefore cannot cause a failover, and this hysteresis also stops a cluster that is flapping during repair from bouncing between states

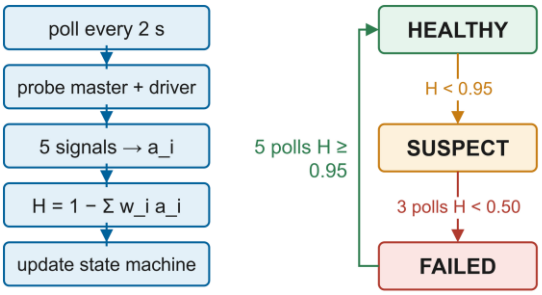


FIGURE 12. Detection workflow and state machine: three polls below 0.50 to fail, five at or above 0.95 to recover.

Different failures land in different states. An unreachable master also counts every worker as lost, giving 0.30 and FAILED after three polls. Full latency anomaly alone gives 0.90 and a long stall alone 0.85, so both mean SUSPECT as slowness alone does not justify moving a job. Baseline B3 uses one rule, unreachable for 30 s, like typical high-availability scripts.

E. CALIBRATION AND FALSE ALARMS

False alarms move jobs for no reason; missed detections leave jobs stuck. Consecutive-poll confirmation, the SUSPECT/FAILED gap and the stall requirement suppress false alarms, while a dead or cut-off cluster always sets the unreachable signal whose weight plus implied worker loss pushes the score below the threshold. The latency signal needs five probes to warm up before it reports anything, so a fault injected in the first ten seconds after the controller starts would be caught only by the other signals; in our experiments the controller was already running, so this never arose.

F. WORKLOAD DIAGNOSTICS AND EARLY FAILURE DETECTION

Because stall and executor loss are per job, the controller sees whether a cluster problem actually hurts the job, which enables the progress-aware rule of Section IX, and operators see the cause in the decision log as the score drops. Predicting failures before they happen [30][35] is not yet part of the design; Section XXI discusses feeding such models into the same interface as baseline-and-deviation detectors do for Kubernetes [73].

IX. FAULT-TOLERANT RECOVERY AND RESILIENCE LAYER

A. RECOVERY ARCHITECTURE

Recovery works at three levels, shown in Fig. 13(a). Spark retries tasks and rebuilds partitions from lineage. The controller restarts a crashed driver on the same healthy cluster, and moves a stalled job off a failed cluster; in both cases the new attempt reloads committed stages. Each level is cheaper than the one above it, and a level is used only when the one below it cannot help, so most failures never reach the controller at all.

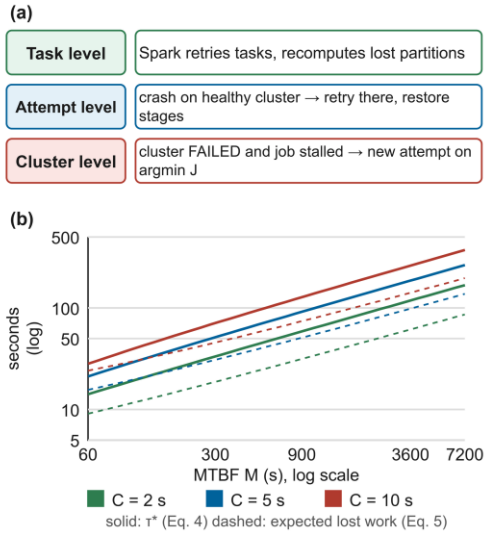


FIGURE 13. (a) Task-, attempt- and cluster-level recovery. (b) τ* from Eq. (4) and expected lost work from Eq. (5) against the MTBF for checkpoint costs of 2, 5 and 10 s.

B. ADAPTIVE CHECKPOINTING

A checkpoint pays a known cost now to lose less later. For checkpoint cost C and mean time between failures M, Young's optimum interval is roughly $\sqrt{(2CM)}$ [43]; Daly's estimate is more accurate when C is not small relative to M [44]:

$$\tau^* = \sqrt{2CM}\left[1 + \frac{1}{3}\sqrt{\frac{C}{2M}} + \frac{1}{9}\frac{C}{2M}\right] - C \quad (4)$$

for $C < 2M$, and $\tau = M$ otherwise. With checkpoints every τ seconds, a failure loses on average half an interval plus the checkpoint in progress:

$$W_{\text{lost}} \approx \frac{\tau}{2} + C \quad (5)$$

The rds library applies this per stage. It tracks the uncheckpointed work W, the summed compute time since the last committed stage, and the cost C as an exponentially weighted average of write times with a 3 s prior, and checkpoints when $W \geq \tau(C, M)$. The controller estimates the MTBF online:

$$\hat{M} = \frac{M_0 n_0 + T_{\text{up}}}{n_0 + N_f} \quad (6)$$

where T_{up} is observed cluster uptime, N_f the number of failures, and $M_0 = 900$ s with weight $n_0 = 1$ a prior that keeps the estimate finite. Failures shrink $\hat{M}$ and τ*, so checkpoints become more frequent; quiet periods do the reverse. Fig. 13(b) plots both quantities. MTBF tracking has been studied for generic systems and stream processors [45][47][49]; we apply it to pipeline stages and share one estimate across clusters. The baselines are never, relying on lineage, and always, checkpointing after every stage.

C. LINEAGE RECOVERY AND STAGE RESTORATION

There is now an attempt which lists the job's commit markers and assigns sequence numbers to its stage names. In the case of linear pipelines, each stage reloads its output and either computes it or leaves it alone since a subsequent stage has been committed. The cost of a reloaded stage is one read of its Parquet output, while that of a computed stage is its usual compute time. Work that is carried out repeatedly is thus limited by the uncheckpointed work at the time of failure, amounting to τ together with the running stage, which is the aim of RCMP [39] and the lineage checkpointing [23] carried out here across clusters.

D. FAILOVER MECHANISMS AND WORKLOAD MIGRATION

The rules for the three modes—none (plain Spark), naive (timeout-based active/passive) and smart (RMC-Spark)—are given in Table 16 in the Appendix. When operating in smart mode, if a driver crashes on a healthy cluster it is retried there once because a crash that passes is not a sufficient reason to switch, and a job only moves to another cluster after 10 seconds without any task progress. In standalone mode the loss of the master does not cause the executors to stop running, so it is still possible for a job to complete; a timeout script would restart the job and thus waste the progress that had already been made, while RMC-Spark records a keep decision.

Figure 14 illustrates the outage workflow. The controller terminates the driver's process group, releases the port, records the failover together with the reason for it, selects a target on the basis of placement cost and then initiates the next attempt. The log entry includes the source and target clusters, the length of time the job had been stalled and the placement score for each candidate, so that the move can be checked later on.

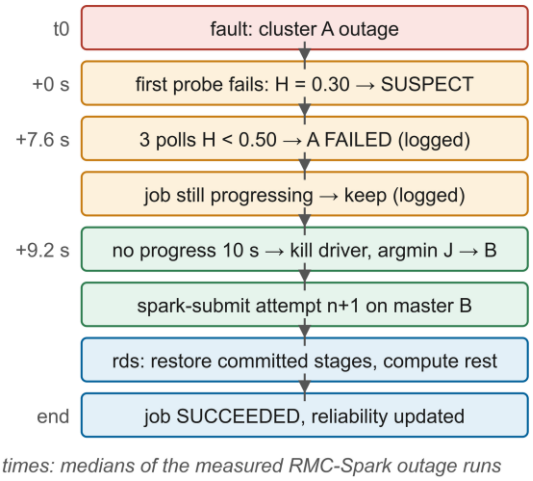


FIGURE 14. Failover workflow for a cluster outage, from detection to stage restoration on the target.

E. CORRECTNESS OF RESUMPTION

A job that has been resumed should produce the same output as one that has not been interrupted. Since the stages are deterministic, carrying out the computation again will yield the same result; outputs are only used when their marker is present, and the markers are recorded after the data, so any outputs that are used are complete; furthermore, the identity of a stage is made up of the job identifier

and the stage name, which means that the outputs cannot be confused. It follows that every output which is consumed in the final attempt is either recomputed or committed based on the same inputs. This is verified in Section XVIII by comparing the quality of the model between resumed and uninterrupted runs.

F. SELF-HEALING STRATEGIES

A cluster which has been repaired remains healthy for five polls and is then readmitted, after which the jobs that were waiting for capacity begin again as soon as one of them becomes eligible. Streaming jobs are restarted from the micro-batch they last committed since both offsets and state are stored in object storage [10]. Self-healing in this case therefore involves the automatic recovery of jobs and the automatic readmission of clusters; the repair of the broken infrastructure itself, for example by restarting containers or by provisioning nodes, is the responsibility of the infrastructure layer, and in the testbed the fault injector restores the clusters between runs.

G. OBSERVED RECOVERY BEHAVIOUR

In the course of the experiments the MTBF estimate yielded a value of τ in the range from 82 s to 112 s, whereas the four ETL stages each required a time between 6 s and 24 s, amounting to about 55 s in total. Since uncommitted work never reached τ, P did not write any checkpoint and, following an outage, the attempt on B had to restart everything, taking between 54 s and 58 s per run, which was close to B3's 62 s. B2 did commit each stage, but it has no failover facility, so it could never make use of those checkpoints: its jobs remained stalled on the failed cluster until the deadline.

It can be seen from equations (4) and (5): since the cost of a checkpoint is only a few seconds and the mean time between failures is about fifteen minutes, a checkpoint only becomes worthwhile after approximately one minute and a half of work, and therefore it is cheaper to restart a shorter pipeline than to take a checkpoint. For longer pipelines and higher failure rates, the importance of being able to resume from a stage is greater, which is the reason why W3 and W2 are the decisive tests in the full matrix.

X. MULTI-CLUSTER COORDINATION AND ORCHESTRATION LAYER

A. COORDINATION MODEL

In As shown in Fig. 15, it is only the controller that is aware of the presence of multiple clusters; the various clusters disregard one another and each driver is aware of only the cluster it currently belongs to. Coordination is carried out in a central and asynchronous manner by means of shared state rather than by messages—specifically, through the job identifier, the commit markers and the controller's own view of each cluster. This approach involves no modifications to Spark and maintains the simplicity of the protocol. The drawback is that the controller itself must be highly available in a production environment, a matter to which we will return in Section XX.

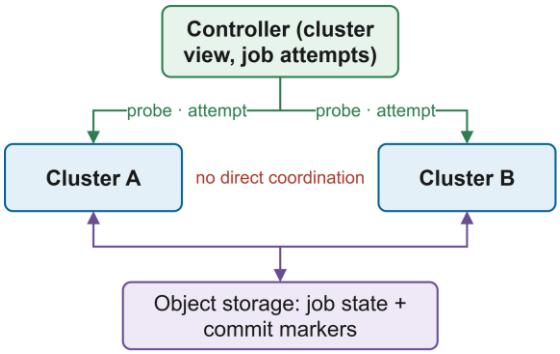


FIGURE 15. Coordination model: clusters never communicate; the controller links them through probes, attempts and shared commit markers.

B. SERVICE DISCOVERY AND CLUSTER REGISTRY

The registry file includes, for each cluster, the master URL, the web endpoint, the names of the fault-injection containers and the expected workers. All that is required to add a cluster is to include an entry, with no need for any changes to the code; in production the registry may be populated by a service-discovery system [14].

C. WORKLOAD PLACEMENT

When a job starts or must move, eligible clusters are ranked by the placement cost:

$$J_k = \alpha L_k + \beta(1 - H_k) + \gamma T_k \qquad (7)$$

The contribution of cluster k is determined by Lk, which is the proportion of cores that are in use, Hk, which is the health score, and Tk, the normalised cost of transferring the job's data and checkpoints to cluster k. The weights are set at α = 0.4, β = 0.5 and γ = 0.1, with health being given the highest priority and idleness the next. Failed clusters as well as the cluster from which the job is being removed are excluded, and in the case of ties determinism is achieved by using the name of the cluster. Tk has a value of zero in the testbed; in a geo-distributed deployment it would depend on the volume of data and the inter-site bandwidth or price [87][88]. Figure 16 illustrates the framework.

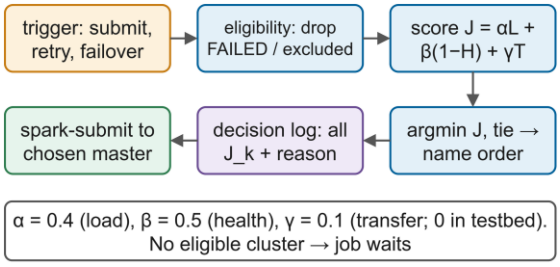


FIGURE 16. Orchestration: eligibility filtering, placement cost and decision logging for placement, retry and failover.

D. ORCHESTRATION POLICIES AND WORKLOAD BALANCING

Placement is considerably simpler than that of Kubernetes schedulers or learned schedulers for cloud jobs [78][116][117]. Since there are only a few clusters, it is cheap to evaluate all of them.

Each decision has to be explainable and a weighted sum is easy to explain. Moreover, since placement has to interact with recovery, it must behave in a predictable way when failures occur. The load term ensures that work is balanced by sending new jobs to the less busy and healthy cluster. A scheduler incorporating security and reliability constraints [84] or a learned policy [28] could take the place of Eq. (7) through the same narrow interface.

E. SYNCHRONISATION AND CONSISTENCY

Since clusters never communicate with one another, problems regarding consistency only occur through the use of shared storage. When a failed attempt is restarted, its executors might still carry out a few writes, for instance after a partition has been healed, but stage writes are idempotent since they produce the same deterministic output in overwrite mode and then set the marker, so the worst case scenario is that the work is duplicated. Structured Streaming protects the location of its own checkpoint [10]. The use of production fencing, such as including attempt numbers in the output paths, would eliminate even the duplication.

F. SCALING TO MORE CLUSTERS

The model can go beyond two clusters without altering the protocol: for each additional cluster one extra probe per poll and one additional candidate for placement are added. The placement cost then spreads the load over a bigger group of nodes, and failover selects from among a number of healthy targets rather than just one passive standby, which is a genuine difference from active and passive designs like B3. However, at a large scale the central controller would become a bottleneck or a single point of failure, so a straightforward way to extend the system is to divide the jobs among several controller replicas, each of which is in charge of some of the jobs but all of which monitors every cluster.

G. COORDINATION PROTOCOL

The protocol consists of five steps when dealing with one job: the controller first submits and inserts attempt zero, then the driver carries out the resilient stages while the controller is polling, the rules set out in Table 16 are applied at each polling event, if an attempt has stalled it is replaced on the selected cluster with the same identifier, and the result updates the reliability statistics of the final cluster. The attempts are given numbers and the state across attempts is stored in object storage, which means that a late write cannot be counted as a successful progress.

H. PLACEMENT DECISIONS IN THE EXPERIMENTS

In every failover the orchestrator chose B with J_B = 0.000, since B was healthy and idle and A was excluded; B3 reached B through its fixed rule. The experiments therefore show that placement works with two clusters, but they never test a real choice between several candidates.

The following example illustrates the way the cost behaves in a situation where there is a choice. Let us suppose that a job has to leave cluster A, that cluster B is in a healthy state (H = 1.0) and is half busy (L = 0.5), while cluster C is SUSPECT with H = 0.9 and is idle. With $\alpha = 0.4$, $\beta = 0.5$ and $\gamma = 0$, Equation (7) yields $\beta = 0.50__0.50__$ and $\beta = 0.51__0.51__$, in which case C is the winner. When β reaches $0.52__0.52__$ the costs are equal and B wins by name; below that value B wins unambiguously. A slightly degraded idle cluster can overcome a busy but healthy one, but a clearly degraded cluster cannot, and both of these values are recorded so that operators may check the choice.

.

XI. EXPLAINABILITY AND RELIABILITY ASSESSMENT LAYER

A. EXPLAINABLE RECOVERY DECISIONS

Since automatic recovery alters both the location and the timing of job executions, operators need to be able to see the reason for this afterwards. In RMC-Spark, the explanation is something that the system produces on its own. Each state transition, submission, retry, failover, keep decision and outcome is added to a JSON decision log. A transition record includes the cluster, the previous and new states, the detector, the health score and the contribution of each signal wi ai,k; a failover record contains the source and target clusters, the stall duration and the placement cost of each candidate. Figure 17 illustrates how these records are generated and used.

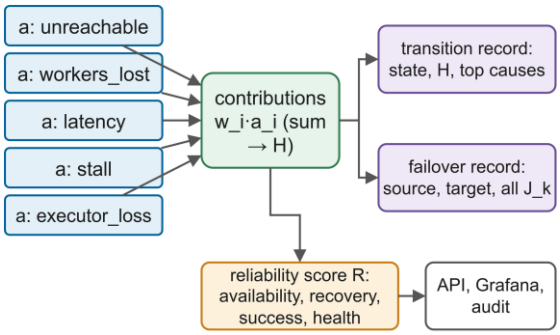


FIGURE 17. Additive health and placement models yield per-factor contributions stored with every decision and aggregated into the reliability score R.

The health model and placement cost are both weighted sums so their explanations are exact in the sense that the contributions sum to the reported score (no surrogate model). This is in contrast to post-hoc explanations of learned detectors and to statistical root cause ranking [74][77]. Log based explanations of cascading microservice failures have been shown to be very useful in terms of guiding the choice of countermeasure [75]; ours are even more direct in that they tell you precisely which signal caused the controller to act.

B. ROOT-CAUSE ANALYSIS

For a failed run you start at the outcome record and then walk backwards through the failover and

transition records to identify the largest contribution prior to the action. A whole cluster outage will show 'unreachable' and 'workers lost' at 0.45 and 0.25 respectively. A stalled job on a cluster that is otherwise reachable will have 'stall' as the primary cause and the stage events will show how much work was re-done. This doesn't in any way replace infrastructure diagnosis with host and network telemetry, it just answers the data science team's question of "why did my job move and what did it cost?"

C. RELIABILITY SCORING

For each cluster the controller maintains uptime and downtime, outage durations, job successes and failures and mean health, all of which are combined into the reliability score of Eq. (8):

$$R_k = \sum_j w_j s_{j,k} \tag{8}$$

where sj,k are four normalised indicators for cluster k and wj their weights (as defined in Table 7). MTTR is normalised against 120s (beyond which recovery is deemed to be fully poor). The score and its components are fed to Prometheus and the API so that the dashboard knows which indicator was responsible for the reliability drop. There are richer service reliability [93] and availability [94] models available; this score is all about transparency rather than prediction.

TABLE 7
RELIABILITY ASSESSMENT INDICATORS

Indicator	Definition	Weight	Interpretation
Availability	Uptime / (uptime + downtime) from state transitions	0.4	Share of time the cluster was usable
Recovery	1 − min(1, MTTR / 120 s)	0.2	Speed of returning to service
Success	Successful attempts / all finished attempts on the cluster	0.2	Whether work placed there completes
Health	Mean health score H over all polls	0.2	Persistent degradation, including gray failures
MTBF estimate	(M_0 n_0 + uptime) / (n_0 + failures), Eq. (6)	—	Input to the checkpoint policy, not part of R

D. WORKED EXAMPLE

In the A cluster outage the first poll sees the master unreachable and no workers reported so the two signals are 0.45 and 0.25 and the health score is 0.30. A becomes SUSPECT and after two further such polls it becomes FAILED, logged as H=0.30 (unreachable=0.45, workers_lost=0.25). 10 seconds later with still no finished task the controller logs a failover from A to B with that reason and the candidates' scores; A is excluded and B is idle and healthy so JB is zero. The next record is attempt one being submitted on B. From those four records the operator can work out what happened without having to look inside the controller at all.

E. LIMITS OF ADDITIVE EXPLANATIONS

An additive model can only explain what it sees. A cause external to the five signals (e.g. a corrupted input file which crashes every time you try to run it) will only manifest as its symptom in the form of repeated driver exits. The weights are also a reflection of our own judgement as to the severity of each signal rather than anything that's been learned from the incident data. For the purposes of making recovery auditable these thresholds are fine, but R should be viewed as a summary of the observed behaviour rather than a calibrated probability of failure.

F. OPERATIONAL TRANSPARENCY

Every threshold, weight and timeout is in one config file and the decisions in Tables 6 and 7 are therefore all derivable from the logged signals. The REST interface provides a list of clusters with their state, health, top causes and reliability, a list of jobs with their attempts and the most recent decisions. There's a command line client to wrap it all. AIOps

research has shown that trust and interpretability are prerequisites to automation [30][65][66] and we provide the exact explanation.

G. DECISION LOG FROM THE EXPERIMENTS

In P's first outage run the controller logged A as SUSPECT with H = 0.30 (unreachable 0.45, workers-lost 0.25) at the fault and 7.3s later FAILED with H = 0.24 as the stall signal went up and there was a keep decision as the driver had made some progress in the 10s. At 10.4s it logged the failover "cluster A FAILED, no progress for 10s; argmin J → B (JB = 0.000)", attempted one on B and then the job completed at 124s. Post all runs R was 0.74 for cluster A (which took all the faults) and 0.98 for B, demonstrating the power of the score in distinguishing between a failing cluster and a stable one.

XII. SECURITY, GOVERNANCE, AND COMPLIANCE LAYER

A. SECURITY CONTROLS

A layer which can stop, start and move jobs is privileged and so its own security is of paramount importance. Figure 18 shows the architecture. The controller has object storage access, the right to submit to every master and in the testbed only Docker Engine access for the purposes of fault injection. In production the fault injection would be removed and the other privileges restricted using service accounts and least privilege policies, combining access control, encryption and integrity protection as per the advice of the big data security frameworks [118][119].

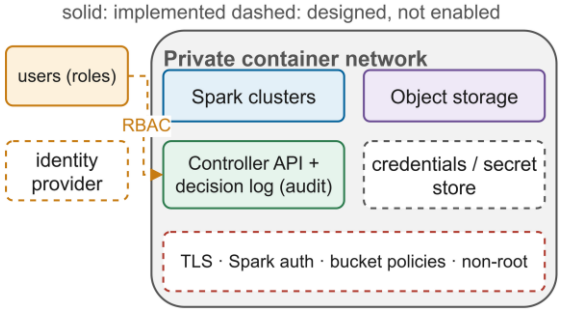


FIGURE 18. Security and governance architecture: implemented controls solid, designed but disabled controls dashed.

We built the testbed for reproducibility rather than for hardening and we state that quite explicitly rather than just leave it as 'production grade'. The implementation includes a private container network, storage credentials, an append only decision log and per job checkpoint namespaces. Spark shared secret authentication, TLS for Spark, storage and API, per job bucket policies, non-root containers and a secret manager have all been designed but disabled. Their status is listed in Table 8.

B. THREAT MODEL

Three of the threats are specific to the cross-cluster layer. An attacker who can submit jobs via the controller can consume resources on all the clusters so submission must be authenticated and rate limited. An attacker who can write to the checkpoint bucket can put a marker next to a forged output which a subsequent attempt would then load, so prefixes can only be written by the job's identity and markers can be signed. An attacker who can trigger failovers (for example by degrading the network on one cluster) could in effect herd the jobs onto a cluster they control; placement constraints and mutual authentication between drivers and executors both mitigate that threat. Big data security surveys recommend encryption and access control at every layer for exactly the same reasons [119].

C. ACCESS MANAGEMENT AND AUDITING

In production the API would be sitting behind the organisation's identity provider with the relevant roles: data scientists would be able to submit and read decisions, platform engineers would have the ability to pause clusters and chaos experiments would have a separate role and require approval. The decision log would serve as the audit trail as every automated action would be timestamped with the job and cluster that it affected and the reason. Chaos engineering for cloud security demonstrates that misconfigurations are best found through deliberate, controlled experiments [96] and with the appropriate approval the same fault injector could be used to test security assumptions

D. DISASTER RECOVERY POLICIES

Each job's RTO is constrained by detection, the stall threshold and driver start-up (Section XVII) and in terms of its RPO in work by τ^* plus one stage. Both of these are predicated on the object store surviving and so in production it must be replicated across failure domains; the single MinIO instance in the testbed is a single point of failure (see Section XX).

E. GOVERNANCE MONITORING AND COMPLIANCE

Governance asks who can use what data for what purpose [112]. Moving a job may involve moving personal data across jurisdictions so the placement cost can assign an infinite transfer cost to any forbidden clusters. The rules are derived from the data catalogues [113]. The evidence is in the decision log (where each attempt actually ran) and in the markers (which attempt produced which output).

TABLE 8
GOVERNANCE AND COMPLIANCE CONTROLS

Control	Purpose	Status in testbed
Private container network	Isolate cluster traffic	Implemented
Object-storage credentials	Authenticate data access	Implemented (lab credentials)
Append-only decision log	Audit automatic actions	Implemented
Per-job checkpoint namespace	Separate job state, attribute outputs	Implemented
Commit markers with attempt number	Provenance of stage outputs	Implemented
Spark shared-secret authentication	Prevent rogue executors and drivers	Designed, not enabled
TLS for Spark, storage and API	Protect data in transit	Designed, not enabled
Least-privilege bucket policies	Restrict jobs to their prefixes	Designed, not enabled
Role-based API access	Separate submit, operate and chaos roles	Designed, not enabled
Non-root containers, secret store	Reduce attack surface	Designed, not enabled
Jurisdiction-aware placement	Keep data in permitted clusters	Designed (via transfer cost)

F. SECURITY OBSERVATIONS FROM THE TESTBED

The testbed taught us two practical lessons. The fault injector requires the Docker Engine socket (i.e. admin access to the host) which is fine for an experiment but absolutely unacceptable for a production controller. The fault injection in a production controller should be a feature of a separate, properly approved chaos engineering tool. The decision log worked perfectly as an audit trail (529 entries over the 36 runs was more than sufficient to reconstruct every failover and every decision). In production the log would go to an append only store that the component in question cannot write to, the lab credentials would be issued and rotated by a secret manager and the object store's own access log would provide a second, independent audit trail.

XIII. EXPERIMENTAL SETUP

A. TESTBED INFRASTRUCTURE

All experiments were run on a single Windows 11 workstation with 12 logical processors and 16GB of memory using Docker Engine 28.3 with WSL2 capped at 12GB (Fig. 19). Each cluster has a 768MB master and two 1.75GB workers (2 cores each, 1280MB). The controller is 2.5GB as it contains the drivers and MinIO, Prometheus and Grafana each take 512MB, 384MB and 256MB respectively. All Spark containers use a single image of Spark 3.5.3 with Java 17 and the Hadoop AWS 3.3.4 connector. One host removes the cloud noise and allows faults to be exactly reproduced [53], though clusters do share resources so the results are all relative to the baselines on the same machine.

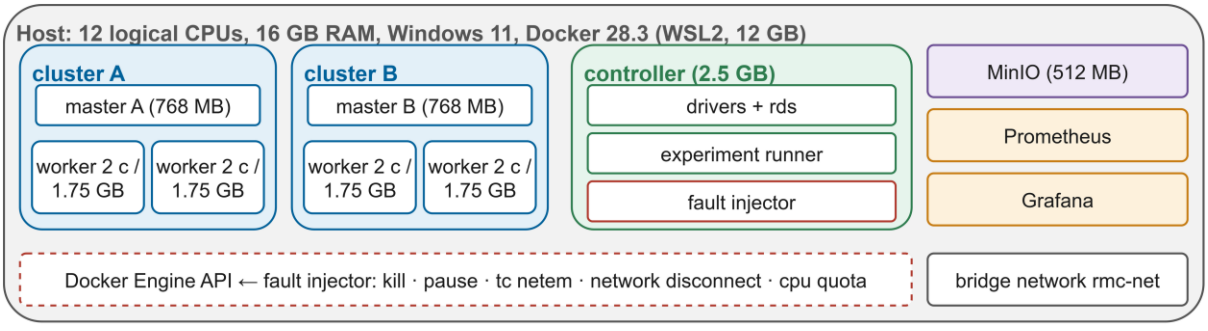


FIGURE 19. Testbed: two Spark clusters, shared MinIO, monitoring, the controller with drivers and runner, and the Docker-based fault injector.

B. DESIGNED AND EXECUTED EVALUATION

The full design is 5 configurations x 4 workloads x 8 fault settings x 3 reps = 480 runs (approx 2 days). For this report we've run a reduced matrix which still gives us every figure and table we need: 4 configurations, W1, 3 fault settings and 3 reps = 36 runs plus two machine learning case studies.

C. WORKLOADS AND DATASETS

W1 is the ETL pipeline from Section VI, W2 is a gradient boosted tree pipeline with a checkpointed model, W3 is an iterative PageRank with long lineage and W4 is a Structured Streaming job with 10s windows. The matrix runs W1 on 1.5M trips; the case studies run W2 on the AI4I 2020 (10k records) and the ULB credit card fraud (284,807 transactions) datasets with mixes similar to the Spark performance studies [12].

D. FAULT INJECTION

A chaos style script [95] operates via the Docker API [97] and supports nine fault types. The matrix employs no fault, executor kill (SIGKILL on one of

the workers' executors, repairable within the cluster) and cluster outage (master and both workers killed). A run starts on A, the fault is applied to the job's cluster at a fixed time and the clusters are restored and verified as healthy prior to the next run.

E. CALIBRATION OF FAULT TIME AND DEADLINE

The fault time was set to 0.4 and the deadline to three times the median clean runtime of the two cold runs (95s and 716s). The warm runs were dramatically faster (from ~299s to ~92s), so in 8 of the 24 fault runs the job actually finished before the fault occurred. These 8 runs were all excluded from the analysis (leaving 28 valid runs with one to three per configuration and fault as per Table 11). Calibrating on warm runs is the first change we'd recommend for the full matrix.

The fault time was set to 0.4 and the deadline to three times the median clean runtime of the two cold runs (95s and 716s). The warm runs were dramatically faster (from ~299s to ~92s), so in 8 of the 24 fault runs the job actually finished before the fault occurred. These 8 runs were all excluded from the analysis (leaving 28 valid runs with one to three per configuration and fault as per Table 11). Calibrating on warm runs is the first change we'd recommend for the full matrix.

F. COMPARED CONFIGURATIONS

TABLE 9
EXPERIMENTAL CONFIGURATION DETAILS

Item	Setting
Host	12 logical CPUs, 16 GB RAM, Windows 11, Docker 28.3 (WSL2, 12 GB)
Spark	3.5.3 standalone, Java 17, Hadoop AWS 3.3.4
Clusters	2 (A, B); per cluster 1 master + 2 workers, 2 cores and 1280 MB per worker
Per job	Driver 512 MB in controller (client mode); executors 1 core / 576 MB; 4 cores
Spark settings	task.maxFailures 4; network.timeout 60 s; heartbeat 10 s; 16 shuffle partitions; AQE on
Storage	MinIO (S3A, path-style), buckets data / checkpoints / results
Monitoring	Controller poll 2 s; Prometheus scrape 5 s; Grafana dashboard
Executed workload	W1 ETL on 1.5 million synthetic trips (four resilient stages)
Case studies	W2 pipeline on AI4I 2020 and ULB credit card; P with checkpoint after every stage
Faults	none, executor kill, cluster outage (fault at 95 s after submission)
Configurations	B1 vanilla, B2 fixed checkpoint, B3 naive HA, P RMC-Spark
Repetitions	3 per cell, runs shuffled within each repetition (seed 2026)
Matrix size	4 × 1 × 3 × 3 = 36 runs, 28 valid (8 finished before the fault time)
Deadline	716 s (three times the calibrated clean runtime)
Designed full matrix	5 configurations × 4 workloads × 8 faults × 3 repetitions = 480 runs (not executed)

G. EXPERIMENTAL PROCEDURE

The runner shuffles the cells in each repetition so that drift favours no configuration and records every run with its decision log, time series and stage events. One script (about 2.5 hours long) runs from testbed start to tables and figures and every number that is reported can be traced back to a raw file.

XIV. EVALUATION METRICS

A. EVALUATION FRAMEWORK

Fig. 20 shows how raw observations become metrics: job-level metrics come from the run records, recovery metrics from the decision log, work metrics from the stage events, and resource metrics from the cluster time series. Table 10 defines each one.

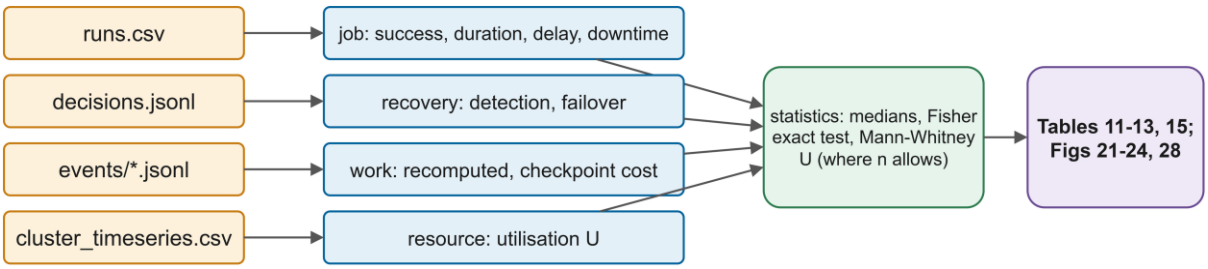


FIGURE 20. Raw observations transformed into job, recovery, work and resource metrics and compared statistically.

B. AVAILABILITY AND RECOVERY METRICS

Steady-state availability follows the classical definition of Eq. (9):

$$A = \frac{\text{MTBF}}{\text{MTBF}+\text{MTTR}} \tag{9}$$

where MTBF and MTTR are the mean times between failures and to repair; per run, availability

is one minus downtime over duration. Recovery time is

$$T_{\text{rec}} = t_{\text{resume}} - t_{\text{fault}} \tag{10}$$

where t_{fault} is the injection time and t_{resume} the submission of the recovering attempt; detection latency, fault to FAILED, is measured separately. Streaming recovery uses Vogel et al.'s criterion of throughput back above 85 per cent of its pre-fault mean for 40 s [53], and stream reliability metrics motivate keeping these measures apart [54].

C. EFFICIENCY METRICS

Fault-free overhead is

$$O = \frac{T_{\text{clean}}^{\text{sys}} - T_{\text{clean}}^{\text{B1}}}{T_{\text{clean}}^{\text{B1}}} \tag{11}$$

where T_{clean}^{sys} and T_{clean}^{B1} are the median clean completion times of the configuration and the vanilla baseline respectively. Recomputed work is the compute time of the stages which were re-run after an earlier attempt had completed them, checkpoint cost is the total write time and utilisation is as per Eq (1).

TABLE 10
METRICS DEFINITION MATRIX

Metric	Definition	Source	Better
Success rate	Share of runs completed before the deadline	Run records	Higher
Completion time	Submission to end of job	Run records	Lower
Fault-induced delay	Completion time minus median clean completion time	Run records	Lower
Downtime	Failed run: end − fault; succeeded run: max(0, delay)	Derived	Lower
Run availability	1 − downtime / duration	Derived	Higher
Detection latency	Fault to FAILED transition of the relevant detector	Decision log	Lower
Recovery time, Eq. (10)	Fault to submission of the recovering attempt	Decision log	Lower
Streaming recovery	Fault to throughput ≥ 85 % of baseline for 40 s	Batch events	Lower
Recomputed work	Compute time of repeated stages	Stage events	Lower
Checkpoint cost	Total checkpoint write time and count	Stage events	Lower
Overhead, Eq. (11)	Relative clean-run slowdown versus B1	Run records	Lower
Utilisation, Eq. (1)	Used / offered core-seconds	Time series	Context

D. METRIC VALIDITY

Each metric has its own known shortcoming. Recovery time stops at the point of resubmission, so the start-up time of the new driver is reported separately in Section XVII. Downtime for successful runs is estimated using the fault-induced delay (which also includes the effect of unrelated slowdowns such as speculative copies fighting for cores) and run availability is a function of run length so it is only compared within a workload. Since all the configurations are treated in the same way these approximations do not introduce any bias to the comparison, but the absolute values should be read with these in mind.

E. STATISTICAL ANALYSIS

Runs are independent so we use two-sided Mann-Whitney U tests, Cliff's delta (with the standard labels: negligible below 0.147, small below 0.33, medium below 0.474, large above) and 95% bootstrap intervals from 5000 resamples. With only one to three valid runs per cell the power of any individual cell is very low so downtime is compared across all faults of a configuration and per-fault results are only described. Completion under outage is a binary outcome so we use the two-sided Fisher exact test. Availability targets are specified as percentiles [29] and cloud availability models use MTBF and MTTR [94] so we use these forms.

XV. RESULTS AND FINDINGS

A. STATUS OF THE EVALUATION AND HYPOTHESES

56 tests check the decision logic (including a simulated outage in which the controller failed over in 16s as opposed to the 30s timeout baseline) and those timings are a check of the logic, not Spark. Before running the matrix of Table 9 we fixed five hypotheses:

H1 (availability): during cluster outages and network partitions P completes a lot more work than either B1 or B2 who are simply unable to leave the failed cluster.

H2 (recovery time): for faults that require failover P recovers faster than B3 as its health detector and stall rule react prior to B3's 30s timeout.

H3 (no unnecessary restarts): under master only failures P completes the jobs without migration whereas B3 will restart them.

H4 (lost work): on the long-lineage workload W3, P and A1 recompute less work after a failure than B1, B2 and B3.

H5 (overhead): in fault-free runs P has the lowest overhead of the three as it only checkpoints when the expected loss is sufficient to justify it.

The reduced matrix ran W1 with B1, B2, B3 and P under no fault, executor kill and cluster outage (three reps each = 36 runs). The 95s fault time was derived from cold runs and as the clean runtime dropped from ~299s to 92s, 8 of the 24 fault runs terminated prior to the fault and were discarded, leaving one to three valid runs per cell (Table 11) i.e. a low power pilot. H3, H4 and A1 were not run.

B. FAULT RECOVERY PERFORMANCE

Fig 21 and Table 11 show the results. For both fault types the median downtime was 52s for P, 614s for B1, 336s for B2 and 90s for B3. Under cluster outage all five valid B1 and B2 runs stalled until the 716s deadline (downtime 614s to 621s), the two P runs and the one valid B3 run moved to cluster B and completed (131s and 112s for P, 134s for B3). Median detection latency was 7.6s for P (7.3s and 8.0s) versus 30.7s for B3 and P resubmitted 8.0s to 10.5s after the fault (30.7s). That 23s lead (the gap between B3's timeout and three 2s polls) had almost no effect on the downtime as P wrote no checkpoints and like B3 simply restarted from stage 1. With only one valid B3 run H2 cannot be tested statistically though the timings are in line with it.

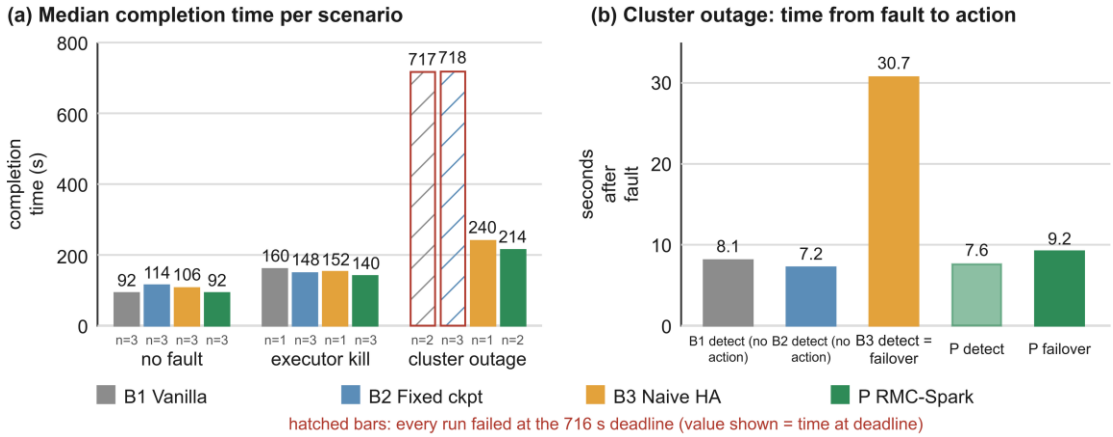


FIGURE 21. (a) Median completion time per fault and configuration with valid-run counts (hatched: failed at the 716 s deadline); (b) time from outage to detection and resubmission.

Spark repaired executor kills within the cluster in all configurations with median times of 68s for B1 (one run), 34s for B2, 46s for B3 (one run) and 48s for P. Again this is consistent with in-cluster recovery. Worker kills and gray failures were left for the full matrix.

TABLE 11
EXPERIMENTAL RESULTS SUMMARY

Configuration	Fault	Runs	Success rate	Median duration (s)	Median downtime (s)	Median detection (s)	Mean recomputed (s)	Mean availability
B1 vanilla	cluster outage	2	0	717	616	n/a	0	0.14
B2 fixed ckpt	cluster outage	3	0	718	620	n/a	0	0.14
B3 naive HA	cluster outage	1	1	240	134	30.7	62.2	0.44
P RMC-Spark	cluster outage	2	1	214	122	7.6	55.7	0.43
B1 vanilla	executor kill	1	1	160	68	n/a	0	0.58
B2 fixed ckpt	executor kill	3	1	148	34	n/a	0	0.79
B3 naive HA	executor kill	1	1	152	46	n/a	0	0.70
P RMC-Spark	executor kill	3	1	140	48	n/a	0	0.65
B1 vanilla	none	3	1	92	0	n/a	0	1
B2 fixed ckpt	none	3	1	114	0	n/a	0	1
B3 naive HA	none	3	1	106	0	n/a	0	1
P RMC-Spark	none	3	1	92	0	n/a	0	1

C. CLUSTER RESILIENCE AND WORKLOAD CONTINUITY

Under cluster outage P completed 2 of 2 valid runs and B3 1 of 1, versus 0 of 2 for B1 and 0 of 3 for B2. Overall success across all valid fault runs was 1.0 for P (5 runs) and B3 (2), 0.5 for B2 (6) and 0.33 for B1 (3). Two-sided Fisher's exact tests give p=0.048 for P versus B1 and B2 combined and p=0.018 for the two failover capable configurations (3 of 3) versus the two single cluster configurations, in support of H1 for the very few runs of outage. P's first outage run logged a keep decision 3 seconds prior to migration and the job was still progressing.

H4 wasn't actually evaluated but the ETL runs do show the policy in action. The MTBF estimate gave a τ^* of 82s to 112s, the four stages took 6s to 24s each (median of 55s total). Uncommitted work never reached τ^* so P wrote no checkpoint and just re-ran each stage on failover: 54s to 58s of recomputed work per outage run (vs 62s for B3). Resumption was therefore never exercised.

D. THROUGHPUT STABILITY FOR STREAMING

Streaming was not run; with offsets and state in object storage a restarted query should be able to resume from its last committed micro-batch [10] as the single cluster studies have shown [52][53].

E. OPERATIONAL AVAILABILITY

Fig 22 shows the mean run availability with 95% bootstrap intervals: 0.57 (0.47 to 0.66) for P, 0.57 (0.44 to 0.70) for B3, 0.46 (0.23 to 0.72) for B2 and 0.29 (0.14 to 0.58) for B1. P and B3 are of course indistinguishable (as expected given that they both restart and differ by about 23s of detection). Against B1 P has significantly lower downtime (large effect, Cliff's delta of −0.73) but this is not significant (p=0.14) with three B1 runs.

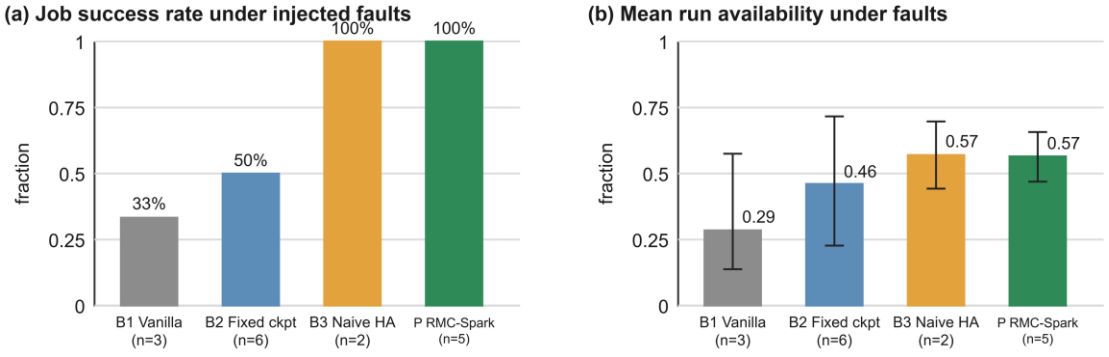


FIGURE 22. Mean run availability under faults with 95 % bootstrap intervals, and success rate per configuration.

F. SYSTEM EFFICIENCY AND RESOURCE UTILISATION

Fig 23 is a P outage run: A runs at full utilisation until the fault at 99s, its health drops from 0.99 to around 0.3, it is resubmitted at 110s and B runs at 1.0 from 120s until completion at 223s. The 21s gap is the obvious recovery time, the rest of the downtime is re-execution. Fault free overhead (eqn (11)) was 0.00 for P and 0.24 for B2, whose per stage checkpoint times were 26s to 45s per run. This supports H5 over B2 only because P wrote no checkpoints at all; A1 was not run.

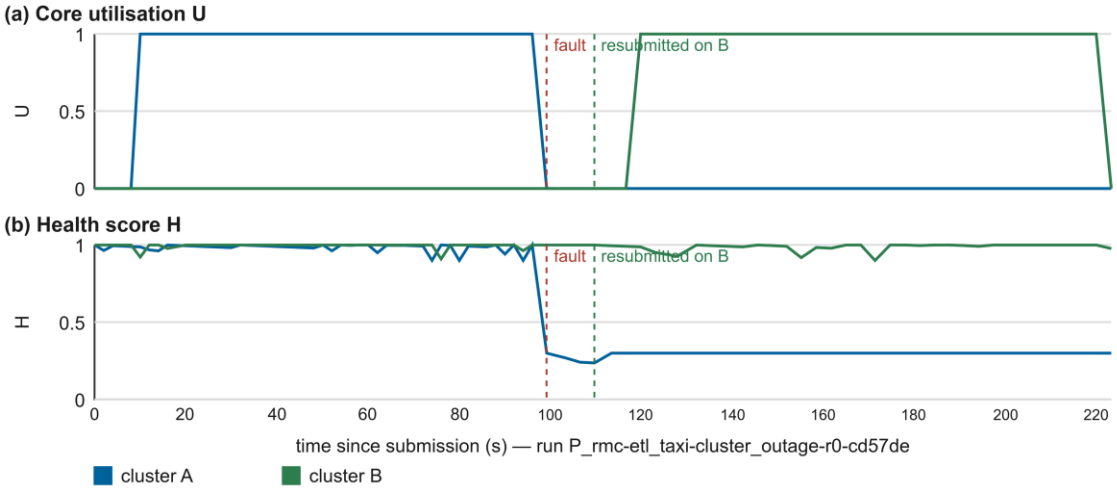


FIGURE 23. Core utilisation (top) and health (bottom) of both clusters in an RMC-Spark outage run, with fault and resubmission marked.

G. SUMMARY OF FINDINGS

P survived every valid outage run whereas B1 and B2 failed all of theirs (H1 supported, p=0.048). It detected outages 23s sooner than B3, though without checkpoints in jobs of about 55s duration the times were very close (122s vs 134s; H2 consistent, untested). P had no measurable overhead over 24% for B2 (H5 supported against B2). The main drawbacks are the lack of valid runs and the very short workload.

XVI. COMPARATIVE ANALYSIS

A. COMPARISON WITH BASELINE DEPLOYMENTS

Table 12 shows the configurations: the top half by design capability, the bottom half by the measured indicators across all the fault runs, compared against vanilla. Fig 24 shows the normalised measured indicators (after min-max) with the outer edge being best.

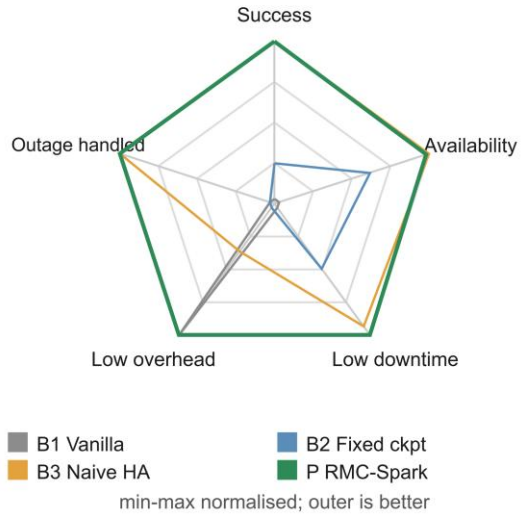


FIGURE 24. Normalised success, availability, downtime, overhead and outage handling per configuration (outer is better).

TABLE 12
COMPARATIVE BENCHMARK ANALYSIS

Property	B1 vanilla	B2 fixed ckpt	B3 naive HA	A1 no adapt	P RMC-Spark
Survives cluster outage	No	No	Yes	Yes	Yes
Resumes from committed stages	No	Same cluster only	No	Yes	Yes
Checkpoint policy	None	Every stage	None	Every stage	Adaptive (Daly)
Failure detection	—	—	Timeout 30 s	Health, 3 polls	Health, 3 polls
Avoids restarting progressing jobs	Yes (no restarts)	Yes (no restarts)	No	Yes	Yes
Explained decisions	No	No	No	Yes	Yes
Success rate under faults	0.33 (n = 3)	0.50 (n = 6)	1.00 (n = 2)	not run	1.00 (n = 5)
Mean availability (95 % CI)	0.29 (0.14–0.58)	0.46 (0.23–0.72)	0.57 (0.44–0.70)	not run	0.57 (0.47–0.66)
Median downtime (s)	614	336	90	not run	52
Clean-run overhead vs B1	0	0.24	0.15	not run	0.00
Downtime vs B1: p (Mann-Whitney)	—	0.91	0.40	not run	0.14
Downtime vs B1: Cliff's delta	—	−0.11 (negligible)	−0.67 (large)	not run	−0.73 (large)

B. ABLATION LOGIC

We picked the configurations so that each pair would isolate one mechanism: B1 vs B2 for checkpointing with no failover, B1 vs B3 for naive failover, B3 vs A1 for detection, progress aware migration and resumption and A1 vs P for the adaptive policy. In the reduced matrix only the first two comparisons (plus a direct B3 vs P) were possible. B1 vs B2: checkpointing with no failover didn't save a single outage run (both failed in the same way, median outage downtime 616s and 620s) and added 24% to the clean runtime. Over all fault runs the difference in downtime was negligible (Cliff's delta −0.11). B1 vs B3: naive failover turned a failed outage run into a completed one (134s downtime) and had a large effect over all faults (Cliff's delta −0.67, p=0.40). B3 vs P: P detected the outage 23s earlier and resubmitted 20-23s earlier but as neither of the short jobs resumed

from a committed stage the difference in outage downtime was only about 12s (122s vs 134s). The A1 comparisons are all still there in the full matrix.

C. COMPARISON WITH SINGLE-CLUSTER AND HIGH-AVAILABILITY PLATFORMS

Single cluster deployments (B1 and B2) lose all jobs on cluster failure no matter what the tuning and fixed checkpointing comes at a cost on every stage without being able to move a job. Standby masters coordinated by ZooKeeper protect the scheduler but not the executors and offer no protection in the case of a zone going away. B3 is scripted high availability which does survive outages but does so by restarting the jobs from scratch and as Table 16 shows can't distinguish between a dead master with working executors and a dead cluster

D. COMPARISON WITH KUBERNETES-BASED ANALYTICS

Running Spark on Kubernetes hands over the lifecycle of the drivers and executors to the orchestrator (which will of course restart the failed pods and reschedule them on healthy nodes [78][79]). That makes the cluster more resilient to node failure and considerably easier to run. Vogel et al.'s benchmark was performed in precisely this configuration [53]. On its own it provides neither failover between Kubernetes clusters nor stage level resumption of a pipeline. RMC-Spark provides the latter: the controller can submit to Kubernetes and the node conditions can be used as health signals. Two Kubernetes clusters didn't fit in the testbed memory so this is left for Section XXI.

E. COMPARISON WITH PUBLISHED BENCHMARKS

The published recovery benchmarks used different hardware so their numbers aren't comparable to ours at all. They found recovery to be very framework dependent [52][53] and engine comparisons do in fact show strong environment effects [106]. Our in-cluster results are consistent with theirs in that once an executor had been killed vanilla Spark did recover in the cluster via task retry and lineage recomputation, completed all of its runs and lost only a few tens of seconds. The novel aspect of this work is that we look at the scenario they didn't (loss of the entire cluster) and we compare the various recovery strategies on top of one engine.

F. PRACTICAL READING OF THE COMPARISON

For practitioners the comparison points to a useful rule of thumb. If cluster outages are infrequent and jobs are cheap to re-run then a single cluster with Spark's native recovery (B1) is sufficient as executor failures are dealt with within the cluster. Fixed checkpointing on a single cluster (B2) is about a quarter of the runtime and provides no benefit at all in the event of the cluster being lost so is difficult to justify on resilience grounds. Scripted failover (B3) already turns outages into finished jobs and is very close to RMC-Spark in the case of short jobs. RMC-Spark provides faster detection, explains the decisions and for long pipelines (where the cost of downtime is significant) provides stage resumption which is the most important feature in that scenario.

XVII. RELIABILITY AND BUSINESS CONTINUITY ASSESSMENT

A. BUSINESS CONTINUITY FRAMEWORK

Business continuity planning poses two questions: how long can a service be down and how much work can be lost? Fig 25 maps these out in the context of RMC-Spark: the recovery time objective (RTO) is the acceptable time between failure and resumption, and the recovery point objective (RPO) is the amount of work that can be lost (measured in work). Cloud service level agreements will of course formalise such objectives and the associated penalties [38].

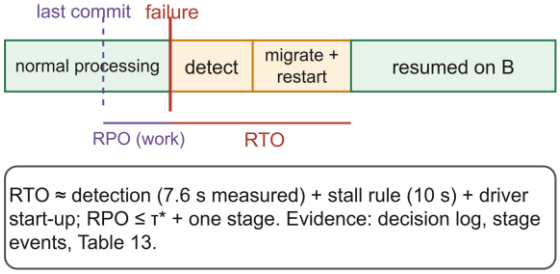


FIGURE 25. RTO and RPO for analytics jobs, the mechanisms that bound them and the evidence used to assess them.

B. RECOVERY OBJECTIVES

The RTO is constrained by detection (3 x 2s polls plus 1.5s timeout), the 10s stall threshold and driver start-up. The first two of those add up to 16s. Start-up took between 13s and 35s across the ETL attempts and 18.3s and 15.0s for P's two attempts on B. P in fact re-submitted between 8s and 10.5s after the fault as detection occurred in the first polls. The RPO is bounded by the uncheckpointed work (kept very close to tau*) and the running stage. For B3 the RTO includes the 30s timeout and the RPO is the entire job as it restarts from scratch; in our short ETL runs P's measured RPO was also essentially the entire job (54s to 58s) as no checkpoint was written.

C. RELIABILITY AND CONTINUITY ASSESSMENT

Table 13 gives the ratings for the configurations (design rows derived from the mechanisms, measured rows from valid fault runs). For B1 and B2 the measured downtime is truncated by the 716s deadline at which the failed jobs were killed and their recomputed work is zero simply because those jobs never ran again; in reality their downtime would have continued until someone resubmitted the job.

TABLE 13
RELIABILITY AND CONTINUITY ASSESSMENT MATRIX

Criterion	B1 vanilla	B2 fixed ckpt	B3 naive HA	A1 no adapt	P RMC-Spark
Design RTO, cluster outage	Unbounded (manual)	Unbounded (manual)	≥ 30 s + start-up	≈ 16 s + start-up	≈ 16 s + start-up
Design RPO, cluster outage	Whole job	Whole job (other cluster)	Whole job	≤ one stage	≤ τ* + one stage
Measured max downtime (s)	618 (deadline)	621 (deadline)	134	not run	131
Measured mean recomputed work (s)	0 (job lost)	0 (job lost)	31	not run	22
Disaster recovery readiness	Low	Low	Medium	High	High
Operational trust (explained actions)	n/a	n/a	Low	High	High

D. DISASTER RECOVERY READINESS

We assess DR readiness via four questions: is there a second site (not B1 or B2), is the resume state available at that site (only A1 and P), is the switch automatic (B3, A1 and P) and has it been tested? The fault injection harness will test the switch over and over again which is what turns a DR plan from a document into a procedure that is actually done in the spirit of chaos engineering [95][99].

E. OPERATIONAL RELIABILITY

Operational reliability is about behaviour over time rather than in a single incident. For each cluster R looks at availability, recovery speed, job success and mean health. The final score was 0.74 for A and 0.98 for B with both clusters healthy at the end (H=1.0). A absorbed every fault as the faults always hit the running job's cluster and the jobs always started on that cluster. The score is a reflection of that imbalance and that's precisely how it would identify a cluster that was chronically unreliable in production.

F. TRUSTWORTHINESS

Trust in automatic recovery is based on three things. Correctness is achieved through idempotent stages and the commit-marker protocol which ensures that if a job is resumed it will produce the same result as if it had run uninterrupted (assuming the stage code is deterministic). Predictability is provided by fixed thresholds and hysteresis so that the behaviour of the system can be reproduced from the configuration. Accountability is provided by the decision log which records every decision and the reason for it. Given that availability is as much a function of repair as it is of failure [94], RMC-Spark improves the repair side of things by making it fast and automatic.

XVIII. INDUSTRY USE CASES AND CASE STUDY EVALUATION

A. BANKING AND FINANCIAL SERVICES

Banks run all sorts of analytics whose value is essentially zero within minutes of them being run (fraud screening, risk aggregation, regulatory reporting etc). Work on internet fraud detection has shown that these types of jobs are very naturally suited to distributed graph and machine learning pipelines [1]. Figure 26 shows a possible deployment: the transactions go into replicated object storage, the scoring stages run as resilient stages on the primary cluster and a cluster in a different availability zone takes over if the primary cluster fails. Our case study runs W2 on the ULB credit card data (the data set is extremely imbalanced with respect to the fraud label) and we inject an outage during training. The outage occurs before the first stage commits so the controller moves the job to B and re-runs both stages. It finishes in 183s as opposed to 138s with no outage. The ROC area is identical to the reference at 0.9868 and the precision-recall area is 0.8428 against 0.8435. That tiny difference indicates that the training stage wasn't fully deterministic in this case (speculative execution will have altered the task scheduling) and as Section IX assumes deterministic stages this would have to be addressed before stage outputs are reused across attempts.

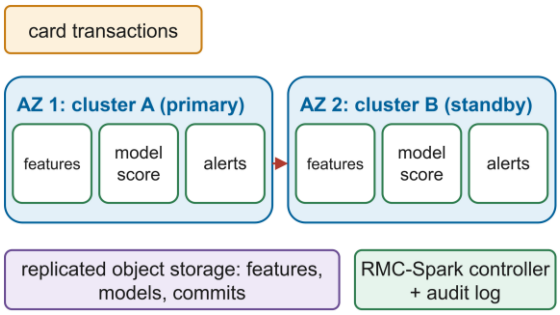


FIGURE 26. Fraud-scoring pipeline on two clusters in different zones with replicated storage and the controller.

B. HEALTHCARE

Healthcare analytics is the combination of clinical, operational and device data for the purposes of treatment and service delivery [120]. Two aspects are important in terms of resilience. Some of the analytics (e.g. capacity planning during a surge) cannot be deferred and patient data may only be processed in certain locations. Section XII's jurisdiction aware placement deals with the latter by eliminating clusters via the transfer cost term.

C. TELECOMMUNICATIONS

Operators are constantly analysing massive volumes of network and customer data for planning, churn prediction and service assurance with governance and architecture being key issues [121]. The relevant pattern is streaming telemetry (our workload W4) where a shared streaming

checkpoint enables a job to continue on another cluster without any data loss.

D. MANUFACTURING

Predictive maintenance uses IoT data and machine learning to predict equipment failure [2]. Figure 27 shows a platform where the sensor streams go into a streaming anomaly job and periodic model training. In our second case study we train W2 on the AI4I 2020 data which contains temperature, rotational speed, torque and tool wear data along with a failure flag. Following an outage the job moved to B and completed in 131s as opposed to 88s; again the fault occurred prior to the first commit so both stages had to re-run. The models in this case were the same with a ROC area of 0.9443 and a precision-recall area of 0.6422.

.

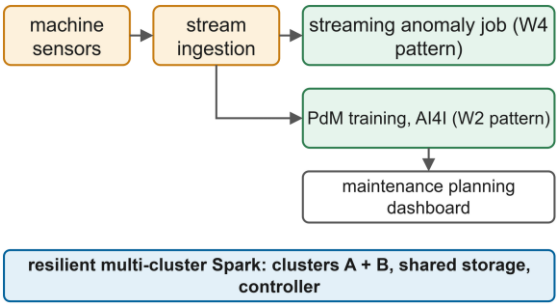


FIGURE 27. Sensor ingestion, streaming anomaly detection and predictive-maintenance training on resilient multi-cluster Spark.

E. CASE STUDY PROTOCOL

Each case study compares a fault-free reference run against a run in which a cluster outage causes migration (and if the feature stage has committed then restoration). We check that the job completed, how much work was re-run and that the model is correct (i.e. matches the reference). It should be as the data split and learner are seeded so any divergence is down to a non-deterministic stage.

F. SMART CITIES AND CYBERSECURITY

Smart city platforms incorporate transport, energy and public service data [122] and spatio-temporal analytics in itself presents a range of resource management challenges [123]. Security analytics has to keep pace with the volume of attacks [124] and deep learning based intrusion detection has been implemented on Spark [125]. Getting the platform lost in the middle of an incident is obviously very expensive. Table 14 details the use cases.

TABLE 14
INDUSTRY USE CASES MATRIX

Domain	Typical analytics	Pattern (workload)	Resilience need	RMC-Spark mechanism
Banking	Fraud scoring, risk aggregation	ML pipeline (W2), streaming (W4)	Minutes-level RTO, audit	Failover, decision log
Healthcare	Capacity planning, cohort analysis	ETL (W1), ML (W2)	Data residency	Jurisdiction-aware placement
Telecommunications	Network telemetry, churn	Streaming (W4)	Continuous processing	Streaming checkpoint in shared storage
Manufacturing	Predictive maintenance	ML (W2), streaming (W4)	Bounded lost training work	Adaptive stage checkpoints
Smart cities	Mobility and energy analytics	ETL (W1), graph (W3)	Long pipelines	Lineage cut by committed stages
Cybersecurity	Intrusion and log analytics	Streaming (W4), ML (W2)	Availability during incidents	Health-driven failover

G. LESSONS FOR ADOPTION

The following three points are worth noting. The greatest benefit is obtained from lengthy and costly pipelines that have to be completed by a deadline, for example those involved in retraining or in aggregating risk at the end of the day, whereas cheap ad hoc queries achieve little. In cases where residency rules apply, placement policies must be in place before automatic failover is allowed, or the recovery process could itself become a compliance incident. Also, the stages have to remain deterministic and idempotent, which requires care when dealing with seeds, timestamps and side effects.

XIX. BUSINESS IMPACT ASSESSMENT

We use a model which separates the measurement from the organisation-specific inputs. Let Nf be the number of annual failures that affect analytics, Dbase the average downtime per failure in the absence of RMC-Spark, Dbase the average downtime per failure in the case of RMC-Spark, Cd the cost of an hour of analytics downtime, and Cextra the annual cost of the second cluster and the controller. The net annual benefit is then

$$\Delta B = N_f(D_{\text{base}} - D_{\text{rmc}})C_d - C_{\text{extra}} \quad (12)$$

D_{base} and D_{rmc} come from Table 11; N_f, C_d and C_{extra} vary by orders of magnitude between, say, a fraud pipeline and a monthly report, so the adopter must supply them. Table 15 structures the assessment and Fig. 28 presents it as a dashboard.

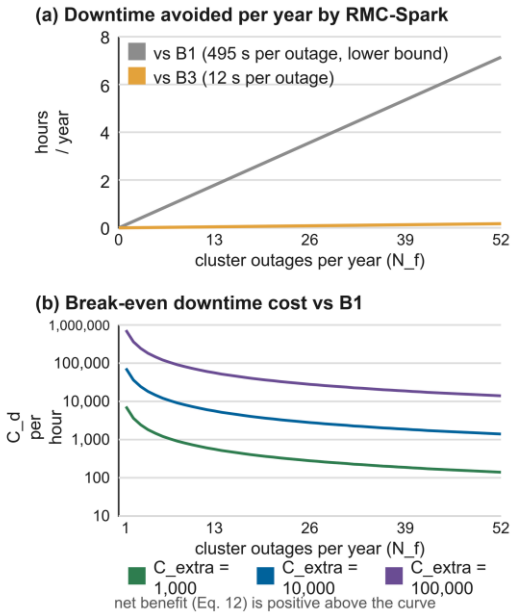


FIGURE 28. (a) Yearly downtime avoided against B1 and B3 by outage rate; (b) break-even downtime cost per hour for Eq. (12) at three values of C_{extra}.

TABLE 15
COST-BENEFIT ANALYSIS

Item	Symbol	Source	Value
Failures per year affecting analytics	N_f	Organisation's incident history	Organisation-specific
Mean downtime per failure, baseline	D_{base}	Experiments (B1 or B3)	616 s for B1 (lower bound, capped by the deadline); 134 s for B3
Mean downtime per failure, RMC-Spark	D_{rmc}	Experiments (P)	122 s
Cost of one hour of analytics downtime	C_d	Business owner	Organisation-specific
Extra capacity and operations per year	C_{extra}	Platform team	Organisation-specific
Recomputation avoided per failure	ΔW	Experiments (stage events)	About 0 s in the ETL runs (no checkpoints written)
Fault-free overhead	O	Experiments, Eq. (11)	0.00 (B2: 0.24)
Net annual benefit	ΔB	Eq. (12)	$N_f \times 0.137\ h \times C_d - C_{extra}$ against B1

Three further effects contribute to the benefit beyond the downtime. Avoided recomputation frees up capacity as the work is reloaded rather than recomputed (i.e. it uses no cluster time). Operators save time as there is no need to go through the process of diagnosing an outage and then resubmitting the jobs by hand and the decision log is much shorter as a result (which in turn shortens the post-incident review). The second cluster is not dead weight in normal operation as the placement cost will distribute the work across both clusters so Cextra provides both throughput and resilience. The offsetting cost of fault-free overhead was zero for P and avoided recomputation was negligible so all the benefit came from the downtime. Against the single cluster baseline P saved at least 495s (0.137h) per outage (a lower bound as the baseline was truncated by the deadline); against scripted failover it was only about 12s. The benefit is positive whenever Nf x 0.137h x Cd is greater than Cextra and SLA penalties provide Cd with a tangible basis [38].

A. SENSITIVITY OF THE BUSINESS CASE

The case is largely a function of failure frequency x downtime cost. If cluster failures are infrequent and downtime is cheap then the extra capacity only makes sense if it's needed for throughput anyway; if the analytics is feeding time-critical decisions then one avoided outage per year can justify it. At two outages per year you avoid 0.27h which equates to an extra cost of 1,000 or 10,000 at about 3,650 or 36,500 per hour of downtime respectively; at twelve outages per year you avoid 1.64h with break-even costs of about 608 and 6,080; at weekly outages you avoid 7.12h with break-even costs of about 140 and 1,400. The baseline downtime is a lower bound so these are conservative figures and with hour-long manual recovery they fall by a factor of ten.

XX. CHALLENGES AND LIMITATIONS

A. CROSS-CLUSTER SYNCHRONISATION

RMC-Spark keeps the clusters in sync solely via shared object storage and a single controller. The design is very simple but that simplicity comes at a cost in that both the shared storage and the single controller are single points of failure. In the testbed this is obvious in that the single MinIO instance and the single controller container are both single points of failure. For production you need storage replicated across failure domains and controller replicas with leader election such that exactly one replica is acting on any given job. Fig 29 shows the limitations and the associated mitigations.

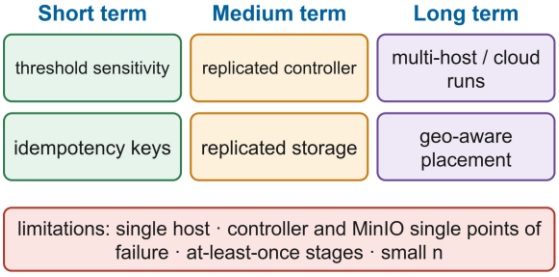


FIGURE 29. Current limitations, their risks and the short-, medium- and long-term mitigations.

B. EXECUTION SEMANTICS

Stages execute at least once. If a migrated attempt overlaps a late write from the old one then both will write the same deterministic output so the results are correct (just with some duplicated work). Stages with external side effects (notifications, writes to a database etc) will need idempotency keys and unseeded randomness will result in different outcomes on resumption.

C. LATENCY OVERHEAD AND NETWORK BOTTLENECKS

Checkpoints cost time and storage bandwidth and forcing each stage to materialise adds a pass over the data. Across sites reading committed stages from a remote store introduces wide area latency which can be the dominant component of job time [89][90]; the transfer term in the placement cost is the hook for that but a single host will never incur it.

D. OPERATIONAL COMPLEXITY AND COST

Running two clusters and a controller is obviously more complex than running a single cluster and the operators have to work from thresholds that we've set by reasoning rather than by tuning, with no sensitivity analysis at this stage. The second cluster does have a cost (although as Section XIX points out it does provide additional throughput).

E. DEPENDENCE ON DETERMINISM AND DATA SIZE

External calls, unseeded randomness and clock dependence all break determinism. Very large stage outputs make checkpoints prohibitively expensive so the adaptive policy will almost never write them and recovery is effectively down to plain lineage recomputation. Very small stages on the other hand add a lot of materialisation overhead for very little gain and a production library would simply merge them.

F. THREATS TO VALIDITY

Internal validity is undermined as the clusters share one host's CPU, memory and disk so a fault on one cluster will impact the other and the absolute timings won't transfer to other machines. External validity is constrained by the fact that the workloads and data sizes are tiny in comparison to actual production analytics and by the fact that network faults are simulated rather than real. The published recovery criteria do support construct validity [53] and with only three repetitions per cell we are limited in what we can conclude so we pool the faults and use non-parametric tests.

G. LIMITATIONS OF THE EXECUTED EVALUATION

Calibrating on the cold runs lost 8 of the 24 fault runs. The one short workload never got to the checkpoint interval so resumption was never exercised and H3 and H4 remain untested. Partitions, master kills and grey failures were all implemented but not measured. A1 wasn't run and the warm-up made the three run overheads very fragile. Each of those has a straightforward fix in the full matrix which is listed in Section XXI.

XXI. FUTURE RESEARCH DIRECTIONS

Fig 30 sets out a research roadmap in four main directions. The first is predictive recovery. The health model is purely reactive but any model that can output a per-cluster anomaly level can provide a signal. Failure predictors trained on cluster traces [35][60] and workload forecasts [64] could be used to allow the controller to checkpoint or migrate prior to a failure (i.e. proactively) just as a proactive cloud scheduler would [62].

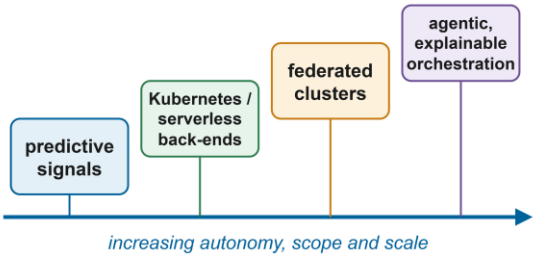


FIGURE 30. Predictive recovery, federated management, new back-ends and agentic, explainable orchestration.

The second is federated cluster management across organisations and clouds. Federated learning trains models across sites without the data ever leaving the site [126], distributed ML has the same kind of placement issues [127] and a federated RMC-Spark would take the residency aware placement of Section XII and extend it across administrative domains. The third is new execution back ends: Spark on Kubernetes and serverless analytics change the way drivers and executors get started [15][16] and the controller's submit and stop

actions could be directed at those without any change to the decision logic. A Kubernetes back end is clearly the next most obvious thing to add to the experimental matrix.

The fourth is agentic orchestration. Language models are being tried for incident analysis and remediation [67] and MLOps and AIOps are being used to close the loop between ops and dev [66]. An advisory agent could read the decision log, propose configuration changes or explain an incident in plain English. The deterministic controller would still have control over every safety-critical action (as Fig 31 illustrates).

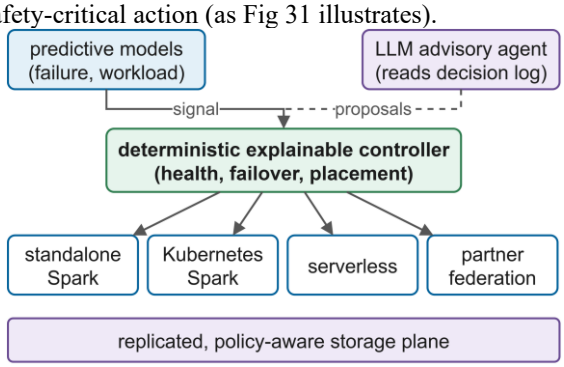


FIGURE 31. Predictive signals and an advisory agent around the deterministic controller, with federated clusters and several back-ends.

Other directions pertain to the controller itself. Learned checkpoint policies could deal with non-memoryless failures (e.g. by looking at time of day or recent anomaly scores) as convergence aware checkpointing does for training [51]. Formal verification could be used to prove that no job which is progressing is ever moved. Price and/or energy could be incorporated into the placement cost as multi-cloud scheduling already does [25][84]. Automatic calibration of the thresholds and weights from incident data would reduce the amount of expert judgement required as noted in Section XI.

A. COMPLETING THE EVALUATION

The most important thing to do is to complete the evaluation that this report has already commenced. The full matrix should be set up to calibrate fault time on warm runs (after a discarded warm-up run), with at least 10 runs per cell. It should include W2 and W3 to allow for resumption and H4 to be tested and master kills for H3, partitions for H1 and A1 to test the policy's own effect. It should be run on two physical hosts or cloud zones to remove the threat of shared host issues.

B. TOWARDS PRODUCTION

Production needs replicated, leader-elected controllers with persistent job state; replicated storage; the security controls as designed switched on with their overheads measured; thresholds calibrated on real incidents (which would also tell you how often cluster failures actually do occur) and that number would be the one that determines the business case.

XXII. CONCLUSION

We were trying to close the gap between the fault tolerance that Spark provides within a cluster and the level of availability that users would expect if the entire cluster failed. RMC-Spark combines resumable program stages with a cross-cluster controller. The rds library makes PySpark stages idempotent and resumable via commit markers and checkpoints them at the Young and Daly interval using an online MTBF estimate. The controller derives its health from five explainable metrics, detects failures using a hysteresis state machine, only moves jobs that have stopped progressing, places them using a readable cost function and logs every decision with the reason for it. Fig 32 maps these contributions to the problems outlined in Section II.

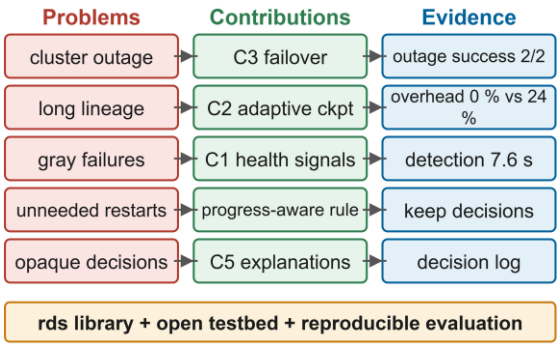


FIGURE 32. Problems from Section II, the components addressing them and the evidence for each.

In 36 runs (28 of which were valid) RMC-Spark detected full outages in around 8s and completed every valid run, whereas the single cluster configurations all failed (H1 supported, p=0.048). Detection was 23s faster than a 30s timeout, but as there were no checkpoints written for jobs of ~55s duration, the downtime was essentially the same as naive failover (fault-free overhead was 0% vs 24% for fixed checkpointing). Both case studies recovered with equivalent or very similar model quality. H3, H4, stage resumption on long workloads and statistical significance are all still to

be determined from the full matrix. The takeaway for the practitioner is that at least on our two cluster testbed, cross cluster resilience for analytics required no changes to the engine at all: a very thin programming library and an explainable controller are sufficient to turn a cluster outage from a failed job into an automatic recovery. The cost of that is something each organisation can assess against the model in Section XIX.

The whole exercise of building the system taught us three things. The most important single decision was to run the drivers outside the clusters (the difference between a job that dies with its cluster and one that can be moved). Separating degradation from failure and a failed cluster from a stopped job was the key to overcoming the principal weakness of timeout based high availability (i.e. the system would go on and restart work that would have completed). Once the models were additive, explaining every decision required no additional machinery at all, which suggests that explainability is a design choice rather than an afterthought. The next bits on the list are predictive signals, Kubernetes and federated back ends and replicated control and storage.

DATA AVAILABILITY

The datasets generated and analysed during the current study are available in the GitHub repository, https://github.com/k745698/rmc-spark: the raw experiment measurements (run records, decision logs, cluster time series and stage events) are in the results/raw directory, and the synthetic taxi-trip and graph datasets are regenerated deterministically by the seeded scripts in jobs/ and scripts/. The public datasets analysed in the case studies are available from the UCI Machine Learning Repository (AI4I 2020 Predictive Maintenance Dataset, https://doi.org/10.24432/C5HS5C) and from the Machine Learning Group of ULB via Kaggle (Credit Card Fraud Detection, https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud); the latter contains only anonymised, PCA-transformed transaction features. The optional NYC TLC Trip Record Data (https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page) can be downloaded with scripts/download_data.py.https://github.com/k745698/rmc-spark

APPENDIX

A. SPARK AND CONTROLLER CONFIGURATION

The repo contains the testbed (docker-compose.yml), the Spark settings (docker/spark-defaults.conf), all of the controller thresholds and weights (configs/clusters.yaml) and the executed and full matrices (experiments/configs) with the values used in this instance being those from Tables 6, 7 and 9.

B. REPRODUCTION STEPS

Following the README: start the testbed with Docker Compose, generate the data, run the smoke test, calibrate the fault times on the machine where the fault is being reproduced, run scripts/run_minimum.ps1 and then regenerate all the tables and figures. The raw results are all in results/ so the analysis can be checked without the testbed and the 56 tests require neither Docker nor Spark.

C. RECOVERY RULES

Table 16 lists the recovery rules the controller applies in each mode.

TABLE 16
RECOVERY RULES PER MODE

Situation	none (B1, B2)	naive (B3)	smart (RMC-Spark)
Driver exits with error, cluster healthy	Job fails	Rerun on passive cluster	Retry on same cluster, then fail over
Driver exits with error, cluster suspect or failed	Job fails	Rerun on passive cluster	Fail over to best cluster
Cluster failed, job still progressing	Job continues	Job restarted elsewhere	Job kept, decision logged
Cluster failed, job stalled 10 s or more	Job hangs until deadline	Restarted after 30 s timeout	Migrated to argmin J
No eligible cluster	Not applicable	Job waits	Job waits, then resumes

D. EXECUTED EXPERIMENT AND CONTROL LOOP

Calibration (experiments/calibrate.py) put in the 95s fault time and 716s deadline into the config. The runner then ran the 36 runs in shuffled order with health checks in between. The raw files are runs.csv, decisions.jsonl (529 decisions), cluster_timeseries.csv and one event file per job. Every 2s the control loop refreshes the driver progress, probes the masters and computes the health score from Eq (3), updates the state machines, applies the rules from Table 16 and exports the metrics to Prometheus.

REFERENCES

[1] H. Zhou, G. Sun, S. Fu, L. Wang, J. Hu, and Y. Gao, "Internet Financial Fraud Detection Based on a Distributed Big Data Approach With Node2vec," *IEEE Access*, vol. 9, pp. 43378–43386, 2021, doi: 10.1109/access.2021.3062467.

[2] R. Rosati et al., "From knowledge-based to big data analytic model: a novel IoT and machine learning based decision support system for predictive maintenance in Industry 4.0," *Journal of Intelligent Manufacturing*, vol. 34, no. 1, pp. 107–121, 2022, doi: 10.1007/s10845-022-01960-x.

[3] M. Zaharia et al., "Apache Spark: a unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016, doi: 10.1145/2934664.

[4] S. Tang, B. He, C. Yu, Y. Li, and K. Li, "A Survey on Spark Ecosystem: Big Data Processing Infrastructure, Machine Learning, and Applications," *IEEE Transactions on Knowledge and Data Engineering*, p. 1, 2020, doi: 10.1109/tkde.2020.2975652.

[5] M. Zaharia et al., "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. 9th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, pp. 15–28, 2012.

[6] S. Isukapalli and S. N. Srirama, "A systematic survey on fault-tolerant solutions for distributed data analytics: Taxonomy, comparison, and future directions," *Computer Science Review*, vol. 53, Art. no. 100660, 2024, doi: 10.1016/j.cosrev.2024.100660.

[7] J. Dean and S. Ghemawat, "MapReduce: simplified data processing on large clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008, doi: 10.1145/1327452.1327492.

[8] V. K. Vavilapalli et al., "Apache Hadoop YARN: yet another resource negotiator," in *Symposium on Cloud Computing*, pp. 1–16, 2013, doi: 10.1145/2523616.2523633.

[9] M. Armbrust et al., "Spark SQL: Relational Data Processing in Spark," in *Proceedings - ACM-SIGMOD International Conference on Management of Data*, pp. 1383–1394, 2015, doi: 10.1145/2723372.2742797.

[10] M. Armbrust et al., "Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark," in *Proceedings - ACM-SIGMOD International Conference on Management of Data*, pp. 601–613, 2018, doi: 10.1145/3183713.3190664.

[11] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, "Apache Flink: Stream and batch processing in a single engine," *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, vol. 36, no. 4, pp. 28–38, 2015.

[12] N. Ahmed, A. L. C. Barczak, T. Susnjak, and M. A. Rashid, "A comprehensive performance analysis of Apache Hadoop and Apache Spark for large scale data sets using HiBench," *Journal Of Big Data*, vol. 7, no. 1, 2020, doi: 10.1186/s40537-020-00388-5.

[13] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at Google with Borg," in *European Conference on Computer Systems*, pp. 1–17, 2015, doi: 10.1145/2741948.2741964.

[14] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016, doi: 10.1145/2890784.

[15] J. Schleier-Smith et al., "What serverless computing is and should become: the next phase of cloud computing," *Communications of the ACM*, vol. 64, no. 5, pp. 76–84, 2021, doi: 10.1145/3406011.

[16] H. B. Hassan, S. A. Barakat, and Q. I. Sarhan, "Survey on serverless computing," *Journal of Cloud Computing Advances Systems and Applications*, vol. 10, no. 1, 2021, doi: 10.1186/s13677-021-00253-7.

[17] R. Hai, C. Koutras, C. Quix, and M. Jarke, "Data Lakes: A Survey of Functions and Systems," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12571–12590, 2023, doi: 10.1109/tkde.2023.3270101.

[18] A. A. Harby and F. Zulkernine, "Data Lakehouse: A survey and experimental study," *Information Systems*, vol. 127, Art. no. 102460, 2024, doi: 10.1016/j.is.2024.102460.

[19] M. Fragkoulis, P. Carbone, V. Kalavri, and A. Katsifodimos, "A survey on the evolution of stream processing systems," *The VLDB Journal*, vol. 33, no. 2, pp. 507–541, 2023, doi: 10.1007/s00778-023-00819-8.

[20] E. Mehmood and T. Anees, "Challenges and Solutions for Processing Real-Time Big Data Stream: A Systematic Literature Review," *IEEE Access*, vol. 8, pp. 119123–119143, 2020, doi: 10.1109/access.2020.3005268.

[21] A. Almeida, S. Brás, S. Sargento, and F. C. Pinto, "Time series big data: a survey on data stream frameworks, analysis and algorithms," *Journal Of Big Data*, vol. 10, no. 1, p. 83, 2023, doi: 10.1186/s40537-023-00760-1.

[22] A. U. Rehman, R. L. Aguiar, and J. P. Barraca, "Fault-Tolerance in the Scope of Cloud Computing," *IEEE Access*, vol. 10, pp. 63422–63441, 2022, doi: 10.1109/access.2022.3182211.

[23] M. Kweun, W.-Y. Lee, G. Kim, J. Hwang, and Y. Lee, "Lineage Checkpoint Approach for Long-lineage Problem in Apache Spark," in *IEEE International Conference on Big Data (BigData)*, pp. 5733–5735, 2020, doi: 10.1109/bigdata50022.2020.9377896.

[24] P. Huang et al., "Gray Failure: The Achilles' Heel of Cloud-Scale Systems," in *Workshop on Hot Topics in Operating Systems*, pp. 150–155, 2017, doi: 10.1145/3102980.3103005.

[25] O. Tomarchio, D. Calcaterra, and G. D. Modica, "Cloud resource orchestration in the multi-cloud landscape: a systematic review of existing frameworks," *Journal of Cloud Computing Advances Systems and Applications*, vol. 9, no. 1, 2020, doi: 10.1186/s13677-020-00194-7.

[26] F. M. Awaysheh, M. Alazab, S. Garg, D. Niyato, and C. Verikoukis, "Big Data Resource Management & Networks: Taxonomy, Survey, and Future Directions," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2098–2130, 2021, doi: 10.1109/comst.2021.3094993.

[27] Z. Wang and A. Aiken, "Efficient Fault Tolerance for Pipelined Query Engines via Write-ahead Lineage," in *Proceedings - International Conference on Data Engineering*, pp. 436–448, 2024, doi: 10.1109/icde60146.2024.00040.

[28] G. Raj, K. Kaur, A. P. Agrawal, P. Kumar, P. K. Pateriya, and A. Punhani, "A meta Proximal policy optimization driven task interaction graph scheduler with predictive resource provisioning and fault-tolerant micro-checkpointing for adaptive spark workflow scheduling in multi-cloud systems," *Engineering Applications of Artificial Intelligence*, vol. 182, Art. no. 115968, 2026, doi: 10.1016/j.engappai.2026.115968.

[29] S. Nastic et al., "SLOC: Service Level Objectives for Next Generation Cloud Computing," *IEEE Internet Computing*, vol. 24, no. 3, pp. 39–50, 2020, doi: 10.1109/mic.2020.2987739.

[30] Y. Li et al., "Predicting Node Failures in an Ultra-Large-Scale Cloud Computing Platform: An AIOps Solution," *ACM Transactions on Software Engineering and*

Methodology, vol. 29, no. 2, pp. 1–24, 2020, doi: 10.1145/3385187.

[31] Q. Lin et al., "Predicting Node failure in cloud service systems," in *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pp. 480–490, 2018, doi: 10.1145/3236024.3236060.

[32] J. Zhang et al., "Minority Disk Failure Prediction Based on Transfer Learning in Large Data Centers of Heterogeneous Disk Systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 9, pp. 2155–2169, 2020, doi: 10.1109/tpds.2020.2985346.

[33] S. Han, P. P. C. Lee, Z. Shen, C. He, Y. Liu, and T. Huang, "StreamDFP: A General Stream Mining Framework for Adaptive Disk Failure Prediction," *IEEE Transactions on Computers*, vol. 72, no. 2, pp. 520–534, 2022, doi: 10.1109/tc.2022.3160365.

[34] K. L. Bawankule, R. K. Dewang, and A. K. Singh, "A classification framework for straggler mitigation and management in a heterogeneous Hadoop cluster: A state-of-art survey," *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 9, pp. 7621–7644, 2022, doi: 10.1016/j.jksuci.2022.02.021.

[35] J. Gao, H. Wang, and H. Shen, "Task Failure Prediction in Cloud Data Centers Using Deep Learning," *IEEE Transactions on Services Computing*, vol. 15, no. 3, pp. 1411–1422, 2020, doi: 10.1109/tsc.2020.2993728.

[36] M. A. Shahid, N. Islam, M. M. Alam, M. S. Mazliham, and S. Musa, "Towards Resilient Method: An exhaustive survey of fault tolerance methods in the cloud computing environment," *Computer Science Review*, vol. 40, Art. no. 100398, 2021, doi: 10.1016/j.cosrev.2021.100398.

[37] M. Kirti, A. K. Maurya, and R. S. Yadav, "Fault-tolerance approaches for distributed and cloud computing environments: A systematic review, taxonomy and future directions," *Concurrency and Computation Practice and Experience*, vol. 36, no. 13, 2024, doi: 10.1002/cpe.8081.

[38] F. Qazi, D. Kwak, F. G. Khan, F. Ali, and S. U. Khan, "Service Level Agreement in cloud computing: Taxonomy, prospects, and challenges," *Internet of Things*, vol. 25, Art. no. 101126, 2024, doi: 10.1016/j.iot.2024.101126.

[39] F. Dinu and T. S. E. Ng, "RCMP: Enabling Efficient Recomputation Based Failure Resilience for Big Data Analytics," in *Proceedings - IEEE International Parallel and Distributed Processing Symposium*, pp. 962–971, 2014, doi: 10.1109/ipdps.2014.102.

[40] C. Li, Q. Cai, and Y. Luo, "Data balancing-based intermediate data partitioning and check point-based cache recovery in Spark environment," *The Journal of Supercomputing*, vol. 78, no. 3, pp. 3561–3604, 2021, doi: 10.1007/s11227-021-04000-2.

[41] J. Shang, Y. Zhang, J. Wang, Z. Wu, and Z. Xiao, "Distributed cache strategy based on LT codes under spark platform," *The Journal of Supercomputing*, vol. 80, no. 11, pp. 16519–16545, 2024, doi: 10.1007/s11227-024-06095-9.

[42] M. He, Z. Fu, and W. Tian, "Optimization of fault tolerance for iterative graph algorithm in spark GraphX based on high performance computing cluster," *CCF Transactions on High Performance Computing*, vol. 7, no. 5, pp. 465–477, 2025, doi: 10.1007/s42514-025-00225-2.

[43] J. W. Young, "A first order approximation to the optimum checkpoint interval," *Communications of the ACM*, vol. 17, no. 9, pp. 530–531, 1974, doi: 10.1145/361147.361115.

[44] J. T. Daly, "A higher order estimate of the optimum checkpoint interval for restart dumps," *Future Generation Computer Systems*, vol. 22, no. 3, pp. 303–312, 2006, doi: 10.1016/j.future.2004.11.016.

[45] P. Sigdel, X. Yuan, and N.-F. Tzeng, "Realizing Best Checkpointing Control in Computing Systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 2, pp. 315–329, 2020, doi: 10.1109/tpds.2020.3015805.

[46] S. Jayasekara, A. Harwood, and S. Karunasekera, "A utilization model for optimization of checkpoint intervals in distributed stream processing systems," *Future Generation Computer Systems*, vol. 110, pp. 68–79, 2020, doi: 10.1016/j.future.2020.04.019.

[47] S. Jayasekara, S. Karunasekera, and A. Harwood, "Optimizing checkpoint-based fault-tolerance in distributed stream processing systems: Theory to practice," *Software Practice and Experience*, vol. 52, no. 1, pp. 296–315, 2021, doi: 10.1002/spe.3021.

[48] Z. Zhang, W. Li, X. Qing, X. Liu, and H. Liu, "Research on Optimal Checkpointing-Interval for Flink Stream Processing Applications," *Mobile Networks and Applications*, vol. 26, no. 5, pp. 1950–1959, 2021, doi: 10.1007/s11036-020-01729-7.

[49] S. M. A. Akber, H. Chen, and H. Jin, "FATM: A failure-aware adaptive fault tolerance model for distributed stream processing systems," *Concurrency and Computation Practice and Experience*, vol. 33, no. 10, 2021, doi: 10.1002/cpe.6167.

[50] C.-Y. Lin, L.-C. Wang, and S.-P. Chang, "Incremental Checkpointing for Fault-Tolerant Stream Processing Systems: A Data Structure Approach," *IEEE Transactions on Emerging Topics in Computing*, vol. 10, no. 1, pp. 124–136, 2020, doi: 10.1109/tetc.2020.2986487.

[51] H. Li, Z. Wang, H. Zhao, M. Zhang, X. Li, and H. Xu, "Convergence-aware optimal checkpointing for exploratory deep learning training jobs," *Future Generation Computer Systems*, vol. 164, Art. no. 107597, 2024, doi: 10.1016/j.future.2024.107597.

[52] G. van Dongen and D. Van den Poel, "A Performance Analysis of Fault Recovery in Stream Processing Frameworks," *IEEE Access*, vol. 9, pp. 93745–93763, 2021, doi: 10.1109/access.2021.3093208.

[53] A. Vogel, S. Henning, E. Perez-Wohlfeil, O. Ertl, and R. Rabiser, "A Comprehensive Benchmarking Analysis of Fault Recovery in Stream Processing Frameworks," in *ACM International Conference on Distributed and Event-Based Systems (DEBS)*, pp. 171–182, 2024, doi: 10.1145/3629104.3666040.

[54] J. Tahir, R. Mayer, C. Doblander, and H.-A. Jacobsen, "How Reliable are Streams? End-to-End Processing-Guarantee Validation and Performance Benchmarking of Stream Processing Systems," *Proceedings of the VLDB Endowment*, vol. 18, no. 3, pp. 585–598, 2024, doi: 10.14778/3712221.3712227.

[55] M. Zaharia, A. Konwinski, A. D. Joseph, R. Katz, and I. Stoica, "Improving MapReduce performance in heterogeneous environments," in *Proc. 8th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, pp. 29–42, 2008.

[56] G. Ananthanarayanan et al., "Reining in the outliers in map-reduce clusters using Mantri," in *Proc. 9th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, pp. 265–278, 2010.

[57] H. Du and S. Zhang, "Hawkeye: Adaptive Straggler Identification on Heterogeneous Spark Cluster With Reinforcement Learning," *IEEE Access*, vol. 8, pp. 57822–57832, 2020, doi: 10.1109/access.2020.2982320.

[58] M. Farhang and F. Safi-Esfahani, "Recognizing MapReduce Straggler Tasks in Big Data Infrastructures Using Artificial Neural Networks," *Journal of Grid Computing*, vol. 18, no. 4, pp. 879–901, 2020, doi: 10.1007/s10723-020-09514-2.

[59] W. Li, D. Liu, K. Chen, K. Li, and H. Qi, "Hone: Mitigating Stragglers in Distributed Stream Processing With Tuple Scheduling," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 8, pp. 2021–2034, 2021, doi: 10.1109/tpds.2021.3051059.

[60] T. N. Tengku Asmawi, A. Ismail, and J. Shen, "Cloud failure prediction based on traditional machine learning and deep learning," *Journal of Cloud Computing Advances Systems and Applications*, vol. 11, no. 1, 2022, doi: 10.1186/s13677-022-00327-0.

[61] H. Bommala, U. M. V., R. Aluvalu, and S. Mudrakola, "Machine learning job failure analysis and prediction model for the cloud environment," *High-Confidence Computing*, vol. 3, no. 4, Art. no. 100165, 2023, doi: 10.1016/j.hcc.2023.100165.

[62] Y. Alahmad, T. Daradkeh, and A. Agarwal, "Proactive Failure-Aware Task Scheduling Framework for Cloud Computing," *IEEE Access*, vol. 9, pp. 106152–106168, 2021, doi: 10.1109/access.2021.3101147.

[63] D. Saxena and A. K. Singh, "OFP-TM: an online VM failure prediction and tolerance model towards high availability of cloud computing environments," *The Journal of Supercomputing*, vol. 78, no. 6, pp. 8003–8024, 2022, doi: 10.1007/s11227-021-04235-z.

[64] D. Saxena, J. Kumar, A. K. Singh, and S. Schmid, "Performance Analysis of Machine Learning Centered Workload Prediction Models for Cloud," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 4, pp. 1313–1330, 2023, doi: 10.1109/tpds.2023.3240567.

[65] P. Notaro, J. Cardoso, and M. Gerndt, "A Survey of AIOps Methods for Failure Management," *ACM Transactions on Intelligent Systems and Technology*, vol. 12, no. 6, pp. 1–45, 2021, doi: 10.1145/3483424.

[66] J. Diaz-de-Arcaya, A. I. Torre-Bastida, G. Zárate, R. Miñón, and A. Almeida, "A Joint Study of the Challenges, Opportunities, and Roadmap of MLOps and AIOps: A Systematic Survey," *ACM Computing Surveys*, vol. 56, no. 4, pp. 1–30, 2023, doi: 10.1145/3625289.

[67] L. Zhang et al., "A Survey of AIOps in the Era of Large Language Models," *ACM Computing Surveys*, vol. 58, no. 2, pp. 1–35, 2025, doi: 10.1145/3746635.

[68] Z. Zamanzadeh Darban, G. I. Webb, S. Pan, C. Aggarwal, and M. Salehi, "Deep Learning for Time Series Anomaly Detection: A Survey," *ACM Computing Surveys*, vol. 57, no. 1, pp. 1–42, 2024, doi: 10.1145/3691338.

[69] K. Choi, J. Yi, C. Park, and S. Yoon, "Deep Learning for Anomaly Detection in Time-Series Data: Review, Analysis, and Guidelines," *IEEE Access*, vol. 9, pp. 120043–120065, 2021, doi: 10.1109/access.2021.3107975.

[70] M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, "Deep learning for anomaly detection in log data: A survey," *Machine Learning with Applications*, vol. 12, Art. no. 100470, 2023, doi: 10.1016/j.mlwa.2023.100470.

[71] D. A. Bhanage, A. V. Pawar, and K. Kotecha, "IT Infrastructure Anomaly Detection and Failure Handling: A Systematic Literature Review Focusing on Datasets, Log Preprocessing, Machine & Deep Learning Approaches and Automated Tool," *IEEE Access*, vol. 9, pp. 156392–156421, 2021, doi: 10.1109/access.2021.3128283.

[72] A. Hrusto, N. b. Ali, E. Engström, and Y. Wang, "Monitoring Data for Anomaly Detection in Cloud-Based Systems: A Systematic Mapping Study," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 4, pp. 1–36, 2025, doi: 10.1145/3744556.

[73] J. G. Almaraz-Rivera, "An Anomaly-based Detection System for Monitoring Kubernetes Infrastructures," *IEEE Latin America Transactions*, vol. 21, no. 3, pp. 457–465, 2023, doi: 10.1109/tla.2023.10068850.

[74] M. Ma, W. Lin, D. Pan, and P. Wang, "ServiceRank: Root Cause Identification of Anomaly in Large-Scale Microservice Architectures," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 3087–3100, 2021, doi: 10.1109/tdsc.2021.3083671.

[75] J. Soldani, S. Forti, L. Roveroni, and A. Brogi, "Explaining Microservices' Cascading Failures From Their Logs," *Software Practice and Experience*, vol. 55, no. 5, pp. 809–828, 2024, doi: 10.1002/spe.3400.

[76] J. Soldani and A. Brogi, "Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey," *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–39, 2022, doi: 10.1145/3501297.

[77] N. Fu, G. Cheng, Y. Teng, G. Dai, S. Yu, and Z. Chen, "Intelligent Root Cause Localization in MicroService Systems: A Survey and New Perspectives," *ACM Computing Surveys*, vol. 57, no. 12, pp. 1–37, 2025, doi: 10.1145/3736755.

[78] C. Carrión, "Kubernetes Scheduling: Taxonomy, Ongoing Issues and Challenges," *ACM Computing Surveys*, vol. 55, no. 7, pp. 1–37, 2022, doi: 10.1145/3539606.

[79] K. Senjab, S. Abbas, N. Ahmed, and A. u. R. Khan, "A survey of Kubernetes scheduling algorithms," *Journal of Cloud Computing Advances Systems and Applications*, vol. 12, no. 1, 2023, doi: 10.1186/s13677-023-00471-1.

[80] Z. Rejiba and J. Chamanara, "Custom Scheduling in Kubernetes: A Survey on Common Problems and Solution Approaches," *ACM Computing Surveys*, vol. 55, no. 7, pp. 1–37, 2022, doi: 10.1145/3544788.

[81] Z. Zhong and R. Buyya, "A Cost-Efficient Container Orchestration Strategy in Kubernetes-Based Cloud Computing Infrastructures with Heterogeneous Resources," *ACM Transactions on Internet Technology*, vol. 20, no. 2, pp. 1–24, 2020, doi: 10.1145/3378447.

[82] S. Tuli, S. R. Poojara, S. N. Srirama, G. Casale, and N. R. Jennings, "COSCO: Container Orchestration Using Co-Simulation and Gradient Based Optimization for Fog Computing Environments," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 1, pp. 101–116, 2021, doi: 10.1109/tpds.2021.3087349.

[83] N. Zhou et al., "Container orchestration on HPC systems through Kubernetes," *Journal of Cloud Computing Advances Systems and Applications*, vol. 10, no. 1, 2021, doi: 10.1186/s13677-021-00231-z.

[84] Q.-H. Zhu, H. Tang, J.-J. Huang, and Y. Hou, "Task Scheduling for Multi-Cloud Computing Subject to Security and Reliability Constraints," *IEEE/CAA Journal of Automatica Sinica*, vol. 8, no. 4, pp. 848–865, 2021, doi: 10.1109/jas.2021.1003934.

[85] Q. Pu et al., "Low Latency Geo-distributed Data Analytics," in *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, pp. 421–434, 2015, doi: 10.1145/2785956.2787505.

[86] T. Z. Emara and J. Z. Huang, "Distributed Data Strategies to Support Large-Scale Data Analysis Across Geo-Distributed Data Centers," *IEEE Access*, vol. 8, pp. 178526–178538, 2020, doi: 10.1109/access.2020.3027675.

[87] L. Chen, S. Liu, and B. Li, "Optimizing Network Transfers for Data Analytic Jobs Across Geo-Distributed Datacenters," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 2, pp. 403–414, 2021, doi: 10.1109/tpds.2021.3093232.

[88] K. Oh, M. Zhang, A. Chandra, and J. Weissman, "Network Cost-Aware Geo-Distributed Data Analytics System," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 6, pp. 1407–1420, 2021, doi: 10.1109/tpds.2021.3108893.

[89] H. Mostafaei, G. Smaragdakis, T. Zinner, and A. Feldmann, "Delay-Resistant Geo-Distributed Analytics," *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, pp. 4734–4749, 2022, doi: 10.1109/tnsm.2022.3192710.

[90] M. Bergui, S. Najah, and N. S. Nikolov, "A survey on bandwidth-aware geo-distributed frameworks for big-data analytics," *Journal Of Big Data*, vol. 8, no. 1, 2021, doi: 10.1186/s40537-021-00427-9.

[91] B. K. Ray, A. Saha, S. Khatua, and S. Roy, "Proactive Fault-Tolerance Technique to Enhance Reliability of Cloud Service in Cloud Federation Environment," *IEEE Transactions on Cloud Computing*, vol. 10, no. 2, pp. 957–971, 2020, doi: 10.1109/tcc.2020.2968522.

[92] Y. Tian, J. Tian, and N. Li, "Cloud reliability and efficiency improvement via failure risk based proactive actions," *Journal of Systems and Software*, vol. 163, Art. no. 110524, 2020, doi: 10.1016/j.jss.2020.110524.

[93] S. Meng, L. Luo, X. Qiu, and Y. Dai, "Service-Oriented Reliability Modeling and Autonomous Optimization of Reliability for Public Cloud Computing Systems," *IEEE Transactions on Reliability*, vol. 71, no. 2, pp. 527–538, 2022, doi: 10.1109/tr.2022.3154651.

[94] P. Maciel et al., "A survey on reliability and availability modeling of edge, fog, and cloud computing," *Journal of Reliable Intelligent Environments*, vol. 8, no. 3, pp. 227–245, 2021, doi: 10.1007/s40860-021-00154-1.

[95] A. Basiri et al., "Chaos Engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016, doi: 10.1109/ms.2016.60.

[96] K. A. Torkura, M. I. H. Sukmana, F. Cheng, and C. Meinel, "CloudStrike: Chaos Engineering for Security and Resiliency in Cloud Infrastructure," *IEEE Access*, vol. 8, pp. 123044–123060, 2020, doi: 10.1109/access.2020.3007338.

[97] J. Simonsson, L. Zhang, B. Morin, B. Baudry, and M. Monperrus, "Observability and chaos engineering on system calls for containerized applications in Docker," *Future Generation Computer Systems*, vol. 122, pp. 117–129, 2021, doi: 10.1016/j.future.2021.04.001.

[98] M. Fogli, C. Giannelli, F. Poltronieri, C. Stefanelli, and M. Tortonesi, "Chaos Engineering for Resilience Assessment of Digital Twins," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 2, pp. 1134–1143, 2023, doi: 10.1109/tii.2023.3264101.

[99] J. Owotogbe, I. Kumara, W.-J. Heuvel, and D. Tamburri, "Chaos Engineering: A Multi-Vocal Literature Review," *ACM Computing Surveys*, vol. 58, no. 7, pp. 1–44, 2025, doi: 10.1145/3777375.

[100] H. Herodotou, Y. Chen, and J. Lu, "A Survey on Automatic Parameter Tuning for Big Data Processing Systems," *ACM Computing Surveys*, vol. 53, no. 2, pp. 1–37, 2020, doi: 10.1145/3381027.

[101] D. Buchaca Prats, F. A. Portella, C. H. A. Costa, and J. L. Berral, "You Only Run Once: Spark Auto-Tuning From a Single Run," *IEEE Transactions on Network and Service Management*, vol. 17, no. 4, pp. 2039–2051, 2020, doi: 10.1109/tnsm.2020.3034824.

[102] Y. Li et al., "Towards General and Efficient Online Tuning for Spark," *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3570–3583, 2023, doi: 10.14778/3611540.3611548.

[103] G. Cheng, S. Ying, and B. Wang, "Tuning configuration of apache spark on public clouds by combining multi-objective optimization and performance prediction model," *Journal of Systems and Software*, vol. 180, Art. no. 111028, 2021, doi: 10.1016/j.jss.2021.111028.

[104] G. Cheng, S. Ying, B. Wang, and Y. Li, "Efficient Performance Prediction for Apache Spark," *Journal of Parallel and Distributed Computing*, vol. 149, pp. 40–51, 2020, doi: 10.1016/j.jpdc.2020.10.010.

[105] S. Shah, Y. Amannejad, and D. Krishnamurthy, "Diaspore: Diagnosing Performance Interference in Apache Spark," *IEEE Access*, vol. 9, pp. 103230–103243, 2021, doi: 10.1109/access.2021.3098426.

[106] M. Dugré, V. Hayot-Sasson, and T. Glatard, "Performance comparison of Dask and Apache Spark on HPC systems for neuroimaging," *Concurrency and Computation Practice and Experience*, vol. 35, no. 21, 2023, doi: 10.1002/cpe.7635.

[107] P. Sawadogo and J. Darmont, "On data lake architectures and metadata management,"

Journal of Intelligent Information Systems, vol. 56, no. 1, pp. 97–120, 2020, doi: 10.1007/s10844-020-00608-7.

[108] S. Ait Errami, H. Hajji, K. Ait El Kadi, and H. Badir, "Spatial big data architecture: From Data Warehouses and Data Lakes to the LakeHouse," *Journal of Parallel and Distributed Computing*, vol. 176, pp. 70–79, 2023, doi: 10.1016/j.jpdc.2023.02.007.

[109] H. Foidl, V. Golendukhina, R. Ramler, and M. Felderer, "Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers," *Journal of Systems and Software*, vol. 207, Art. no. 111855, 2023, doi: 10.1016/j.jss.2023.111855.

[110] I. Taleb, M. A. Serhani, C. Bouhaddioui, and R. Dssouli, "Big data quality framework: a holistic approach to continuous quality management," *Journal Of Big Data*, vol. 8, no. 1, 2021, doi: 10.1186/s40537-021-00468-0.

[111] A. H. Chillón, M. Klettke, D. S. Ruiz, and J. G. Molina, "A Generic Schema Evolution Approach for NoSQL and Relational Databases," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 2774–2789, 2024, doi: 10.1109/tkde.2024.3362273.

[112] M. Janssen, P. Brous, E. Estevez, L. S. Barbosa, and T. Janowski, "Data governance: Organizing data for trustworthy Artificial Intelligence," *Government Information Quarterly*, vol. 37, no. 3, Art. no. 101493, 2020, doi: 10.1016/j.giq.2020.101493.

[113] S. Nadal, P. Jovanovic, B. Bilalli, and O. Romero, "Operationalizing and automating Data Governance," *Journal Of Big Data*, vol. 9, no. 1, p. 117, 2022, doi: 10.1186/s40537-022-00673-5.

[114] M. Usman, S. Ferlin, A. Brunstrom, and J. Taheri, "A Survey on Observability of Distributed Edge & Container-Based Microservices," *IEEE Access*, vol. 10, pp. 86904–86919, 2022, doi: 10.1109/access.2022.3193102.

[115] B. Li et al., "Enjoy your observability: an industrial survey of microservice tracing and analysis," *Empirical Software Engineering*, vol. 27, no. 1, p. 25, 2021, doi: 10.1007/s10664-021-10063-9.

[116] Y. Kang, L. Pan, and S. Liu, "Job scheduling for big data analytical applications in clouds: A taxonomy study," *Future Generation Computer Systems*, vol. 135, pp. 129–145, 2022, doi: 10.1016/j.future.2022.04.035.

[117] Y. Sanjalawe, S. Al-E'mari, S. Fraihat, and S. Makhadmeh, "AI-driven job scheduling in cloud computing: a comprehensive review," *Artificial Intelligence Review*, vol. 58, no. 7, 2025, doi: 10.1007/s10462-025-11208-8.

[118] H. E. Alhazmi, F. E. Eassa, and S. M. Sandokji, "Towards Big Data Security Framework by Leveraging Fragmentation and Blockchain Technology," *IEEE Access*, vol. 10, pp. 10768–10782, 2022, doi: 10.1109/access.2022.3144632.

[119] S. Lu, J. Zheng, Z. Cao, Y. Wang, and C. Gu, "A survey on cryptographic techniques for protecting big data security: present and forthcoming," *Science China Information Sciences*, vol. 65, no. 10, 2022, doi: 10.1007/s11432-021-3393-x.

[120] K. Batko and A. Ślęzak, "The use of Big Data Analytics in healthcare," *Journal Of Big Data*, vol. 9, no. 1, p. 3, 2022, doi: 10.1186/s40537-021-00553-4.

[121] M. Z. Kastouni and A. Ait Lahcen, "Big data analytics in telecommunications: Governance, architecture and use cases," *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 6, pp. 2758–2770, 2020, doi: 10.1016/j.jksuci.2020.11.024.

[122] M. Fugini, J. Finocchi, and P. Locatelli, "A Big Data Analytics Architecture for Smart Cities and Smart Companies," *Big Data Research*, vol. 24, Art. no. 100192, 2021, doi: 10.1016/j.bdr.2021.100192.

[123] H. Liang, Z. Zhang, C. Hu, Y. Gong, and D. Cheng, "A Survey on Spatio-Temporal Big Data Analytics Ecosystem: Resource Management, Processing Platform, and Applications," *IEEE Transactions on Big Data*, vol. 10, no. 2, pp. 174–193, 2023, doi: 10.1109/tbdata.2023.3342619.

[124] F. Ullah and M. A. Babar, "On the scalability of Big Data Cyber Security Analytics systems," *Journal of Network and Computer Applications*, vol. 198, Art. no. 103294, 2021, doi: 10.1016/j.jnca.2021.103294.

[125] M. Haggag, M. M. Tantawy, and M. M. S. El-Soudani, "Implementing a Deep Learning Model for Intrusion Detection on Apache Spark Platform," *IEEE Access*, vol. 8, pp. 163660–163672, 2020, doi: 10.1109/access.2020.3019931.

[126] A. Imteaj, U. Thakker, S. Wang, J. Li, and M. H. Amini, "A Survey on Federated Learning for Resource-Constrained IoT Devices," *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 1–24, 2021, doi: 10.1109/jiot.2021.3095077.

[127] J. Verbraeken, M. Wolting, J. Katzy, J. Kloppenburg, T. Verbelen, and J. S. Rellermeyer, "A Survey on Distributed Machine Learning," *ACM Computing Surveys*, vol. 53, no. 2, pp. 1–33, 2020, doi: 10.1145/3377454.



Dr. M. Rajasekhara Babu received the B.Tech. Degree in Electronics and Communication Engineering from Sri Venkateswara (S.V.) University, Andhra Pradesh, the M.Tech. degree in Computer Science and Engineering from REC Calicut (now NIT Calicut), Kerala, and the Ph.D. degree from VIT University, Vellore, India.

He worked with Intel E.C., India from 2000 to 2002 before joining VIT University in 2002, where he currently serves as a Professor in the School of Computer Science and Engineering. He was instrumental in establishing advanced laboratories in Multi-Core Architectures, Embedded Systems, and Internet of Things (IoT) with support from Intel India. He has collaborated with NUS, NTU Singapore, and the University of Malaya, Malaysia. He has published 10+ patents, guided around 10+ Ph.D. scholars, authored 100+ research publications, co-authored books, and edited international conference proceedings published by Allied Publishers, Macmillan, and Springer. His research interests include IoT, High-Performance Computing (HPC) Artificial Intelligence, and Data Analytics.

Prof. Rajasekhara Babu is associated with the Computer Society of India (CSI), IEEE, and IAENG. He has held several academic and administrative leadership positions at VIT and received the Best Teacher Award (2005-2006) and the Best E-Content Development Award (2007) for his contributions to teaching, research, and academic excellence.



Mudavath Narasimha Naik (Reg. No. 26MML0054) is a postgraduate student in the School of Computer Science and Engineering, VIT, Vellore, India. Research interests: anomaly detection and AIOps. Wrote Sections VIII, XI, XIV, XV, XIX and XXI.



Vana Shivaramakrishna Kumar (Reg. No. 26MML0006) is a postgraduate student in the School of Computer Science and Engineering, VIT, Vellore, India. Research interests: big data engineering and Spark performance. Wrote Sections II, IV, VI, VII, XIII and XVI and the Appendix.



Vinayak C (Reg. No. 26MML0047) is a postgraduate student in the School of Computer Science and Engineering, VIT, Vellore, India. Research interests: cloud security and data governance. Wrote Sections I, III, XII, XVII and XVIII.



K V Kartik (Reg. No. 26MML0015) is a postgraduate student in the School of Computer Science and Engineering, VIT, Vellore, India. Research interests: distributed systems and fault tolerance. Team point of contact; wrote the abstract and Sections V, IX, X, XX and XXII.